S1

S2

2

QUESTION DESCRIPTION

Question 2

MovieDB is an app that allows users to explore movies, submit ratings, and share reviews.

The watchlist feature helps users save and manage movies they want to watch later. Currently, this feature is not functioning correctly on the backend, affecting usability and engagement.

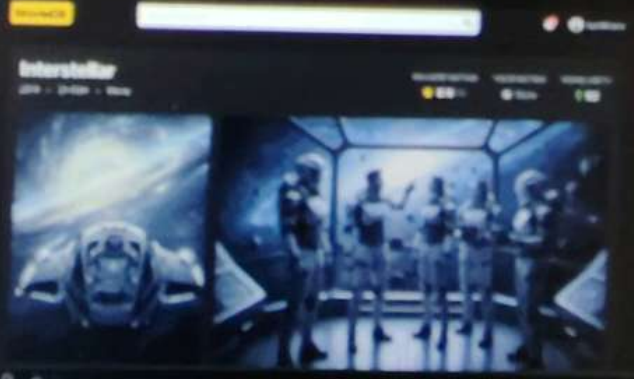
Issue Summary:
Users can add movies to their watchlist, but these movies do not appear on the Watchlist page. Fix the backend of the watchlist feature.

Steps to Reproduce:

- Log in with the following test credentials:

```
Email: sam.rivera@email.com
Password: Password@123
```

- Select a movie from the list.
- Add the selected movie to the watchlist.



README.md

MovieDB(MER

Overview

MovieDB is an online app that

The watchlist feature helps us
is not functioning correctly on

Expected API Behav

GET /api/watchlist/

Purpose Retrieves all movies

Auth: Required (Bearer token)

Success Responses (200)


```
[
  {
    "_id": "movie_id"
    "title": "The Sh
    "year": 1994,
    "duration": "2h
    "rating": 9.3,
    "popularity": 98
    "genre": ["Drama"
    "description": "
    "director": "Fra
    "writers": ["Step
    "stars": ["Tim Ro
    "type": "movie",
    "createdAt": "202
    "updatedAt": "202
  }
]
```

README.md

Markdown Preview

AI Assistant

MovieDB(MERN): User Watchlist

Overview

MovieDB is an online app that allows users to explore movies, submit ratings, and share reviews.

The watchlist feature helps users save and manage movies they want to watch later. Currently, this feature is not functioning correctly on the backend and your task is to fix it.

Expected API Behavior

GET /api/watchlist/

Purpose Retrieves all movies in the authenticated user's watchlist

Auth: Required (Bearer token)

Success Response (200)


```
[
  {
    "_id": "movie_id",
    "title": "The Shawshank Redemption",
    "year": 1994,
    "duration": "2h 22m",
    "rating": 9.3,
    "popularity": 90,
    "genre": ["Drama", "Crime"],
    "description": "Two imprisoned men bond over a number of years...",
    "director": "Frank Darabont",
    "writers": ["Stephen King", "Frank Darabont"],
    "stars": ["Tim Robbins", "Morgan Freeman"],
    "type": "movie",
    "createdAt": "2024-01-01T00:00:00.000Z",
    "updatedAt": "2024-01-01T00:00:00.000Z"
  }
]
```


Ask the AI As

ASUS Vivobook

58 min 52 sec

S1

S2

2

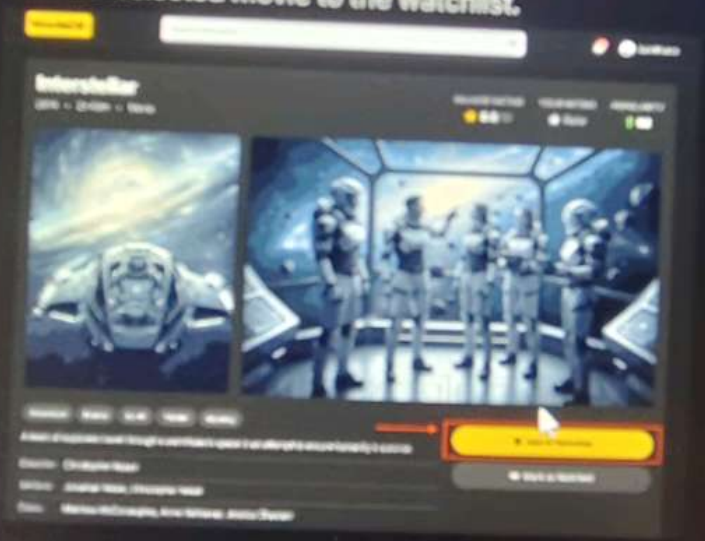
QUESTION DESCRIPTION

Steps to Reproduce:

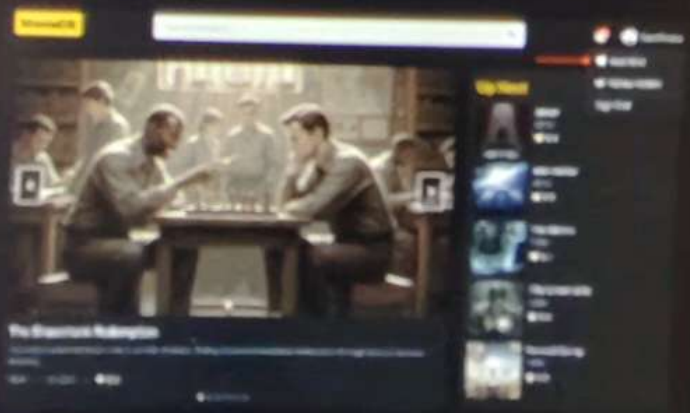
- Log in with the following test credentials:

```
Email: sam.rivera@email.com
Password: Password@123
```

- Select a movie from the list.
- Add the selected movie to the watchlist.



- Click on the user profile present in the navbar and select watchlist.



- The movie should appear here, but currently, there are

README.md

MovieDB(ME

Overview

MovieDB is an online app t

The watchlist feature helps
is not functioning correctly

Expected API Beha

GET /api/watchlist/

Purpose Retrieves all movie

Auth: Required (Bearer toke

Success Responses (200)


```
[
  {
    "_id": "movie_1
    "title": "The S
    "year": 1994,
    "duration": "2h
    "rating": 9.3,
    "popularity": 9
    "genre": ["Dram
    "description":
    "director": "Fr
    "writers": ["St
    "stars": ["Tim
    "type": "movie"
    "createdAt": "2
```

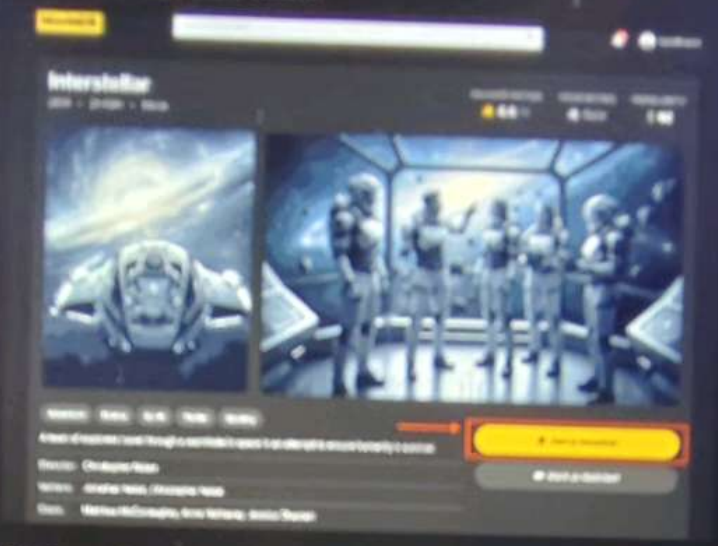
58 min 32 sec

S1

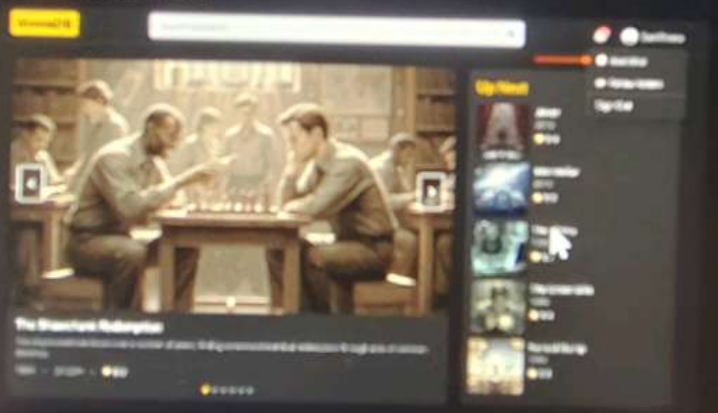
S2

2

QUESTION DESCRIPTION



- Click on the user profile present in the navbar and select watchlist.



- The movie should appear here, but currently, there are none.
- Users should be able to remove movies from the watchlist by either hovering over a movie on the Watchlist page and clicking the delete icon, or by toggling the "Add to Watchlist" button on the movie detail page.



README.md

MovieDB(

Overview

MovieDB is an online

The watchlist feature

is not functioning co

Expected API

GET /api/watchli

Purpose Retrieves all

Auth: Required (Bear

Success Responses

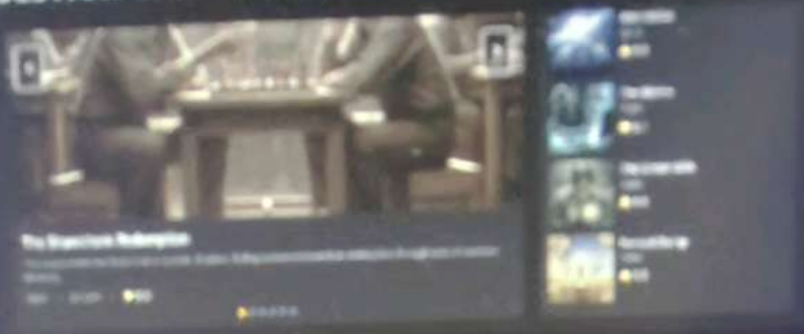

```
[
  {
    "_id": "
    "title": "
    "year": 19
    "duration"
    "rating":
    "popularity
    "genre": ["
    "descriptio
    "director":
    "writers":
    "stars": ["
    "type": "mo
    "createdAt"
    "updatedAt"
  }
```

S1

S2

2

QUESTION DESCRIPTION



- The movie should appear here, but currently, there are none.
- Users should be able to remove movies from the watchlist by either hovering over a movie on the Watchlist page and clicking the delete icon, or by toggling the "Add to Watchlist" button on the movie detail page.



Additional Requirements:

To prevent spam or automated activity, the system must apply rate limiting. Each user can only add or remove movies up to 10 times within any 2-minute period.

Refer to the README.md file for more details.

Note: The acceptance criteria for this task require that your solution pass all predefined unit tests. Use failing test cases to guide debugging.

MovieD

Overview

MovieDB is an o

The watchlist fe
is not functionin

Expected A

GET /api/wato

Purpose Retrieve

Auth: Required (B

Success Respons

```
[
    {
        "_id": "
        "title":
        "year":
        "duratio
        "rating"
        "popular
        "genre":
        "descript
        "director
        "writers"
        "stars":
        "type": "
        "createdA
        "updatedA
    }
]
```

README.md

Markdown Preview AI Assistant

```
        "popularity": 98,
        "genre": ["Drama", "Crime"],
        "description": "Two imprisoned men bond over a number of years...",
        "director": "Frank Darabont",
        "writers": ["Stephen King", "Frank Darabont"],
        "stars": ["Tim Robbins", "Morgan Freeman"],
        "type": "movie",
        "createdAt": "2024-01-01T00:00:00.000Z",
        "updatedAt": "2024-01-01T00:00:00.000Z"
    }
]
```

GET /api/watchlist/check/:movieId

Purpose Checks if a specific movie is in the user's watchlist

Auth: Required (Bearer token)

Success Responses (200)

```
{
    "inWatchlist": true
}
```

Error Response:

Invalid movie ID (400):

```
{
    "message": "Invalid movie ID format"
}
```

POST /api/watchlist/add

Purpose Adds a movie to the user's watchlist

Auth: Required (Bearer token) + Rate limiting (10 add/remove operations per 2 minutes combined)

Request Body

ASUS Vivobook

README.md

Markdown Preview

AI Assistant

POST /api/watchlist/add

Purpose Adds a movie to the user's watchlist

Auth: Required (Bearer token) + Rate limiting (10 add/remove operations per 2 minutes combined)

Request Body

```
{
  "movieId": "movie_id"
}
```

Success Responses (201)

```
{
  "message": "Movie added to watchlist successfully"
}
```

Error Responses:

Missing movie ID (400):

```
{
  "message": "Movie ID is required"
}
```

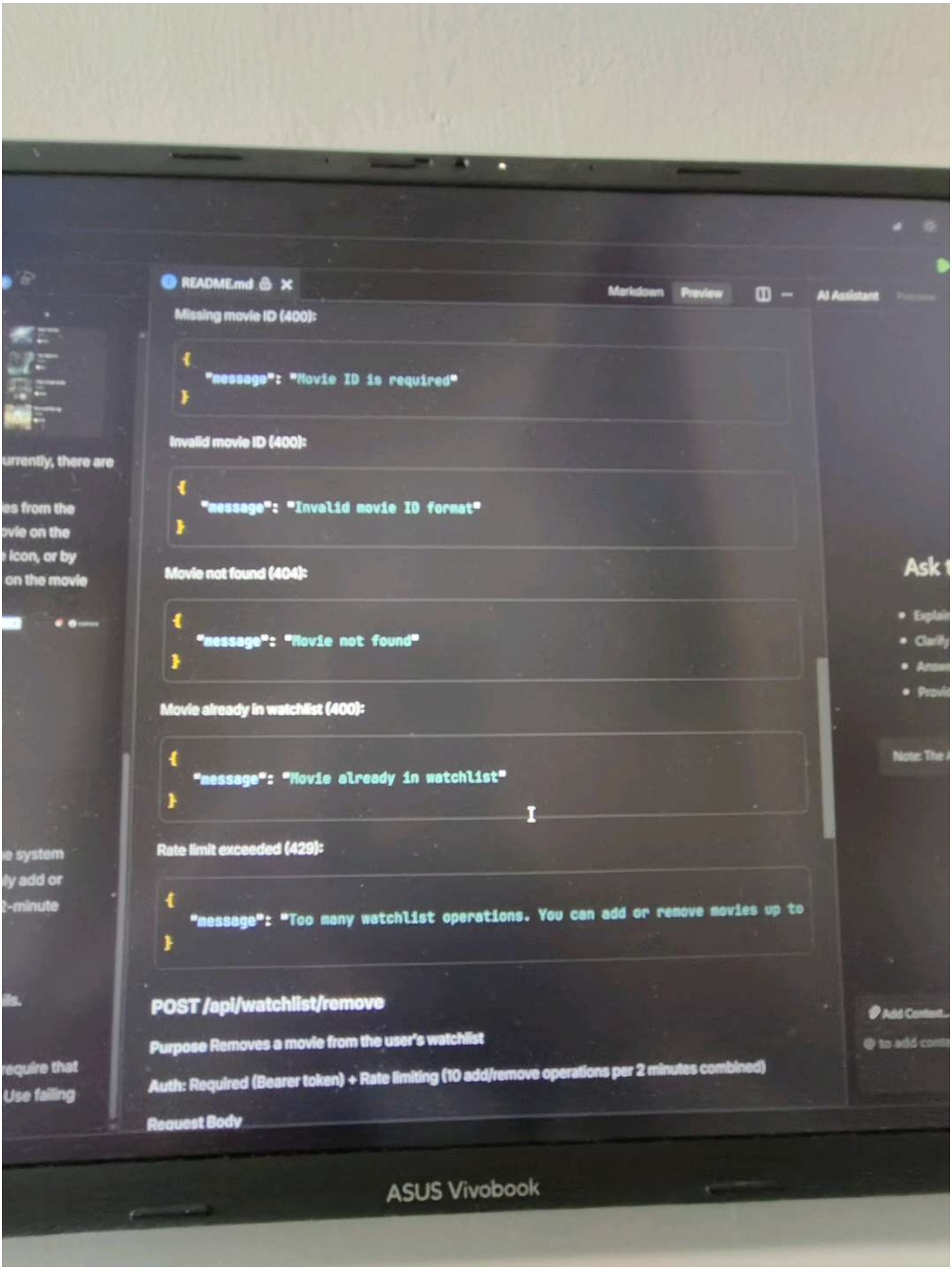
Invalid movie ID (400):

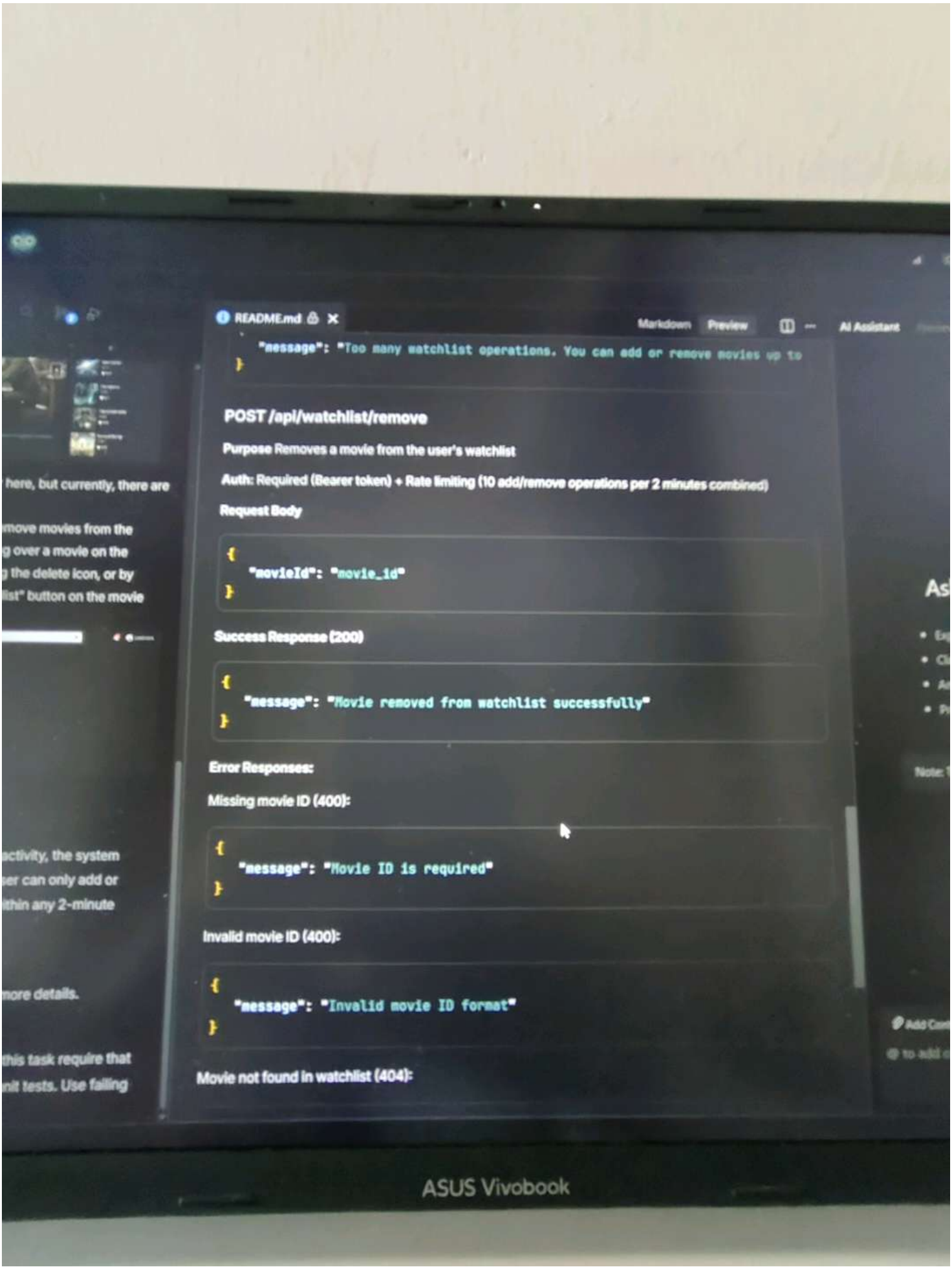
```
{
  "message": "Invalid movie ID format"
}
```

Movie not found (404):

```
{
  "message": "Movie not found"
```

ASUS Vivobook


README.md
Markdown
Preview
AI Assistant
Missing movie ID (400):
{
"message": "Movie ID is required"
}
Invalid movie ID (400):
{
"message": "Invalid movie ID format"
}
Movie not found (404):
{
"message": "Movie not found"
}
Movie already in watchlist (400):
{
"message": "Movie already in watchlist"
}
Rate limit exceeded (429):
{
"message": "Too many watchlist operations. You can add or remove movies up to
}
POST /api/watchlist/remove
Purpose Removes a movie from the user's watchlist
Auth: Required (Bearer token) + Rate limiting (10 add/remove operations per 2 minutes combined)
Request Body
ASUS Vivobook


README.md
Markdown
Preview
AI Assistant
"message": "Too many watchlist operations. You can add or remove movies up to
}
POST /api/watchlist/remove
Purpose Removes a movie from the user's watchlist
Auth: Required (Bearer token) + Rate limiting (10 add/remove operations per 2 minutes combined)
Request Body
{
"movieId": "movie_id"
}
Success Response (200)
{
"message": "Movie removed from watchlist successfully"
}
Error Responses:
Missing movie ID (400):
{
"message": "Movie ID is required"
}
Invalid movie ID (400):
{
"message": "Invalid movie ID format"
}
Movie not found in watchlist (404):
here, but currently, there are
move movies from the
g over a movie on the
the delete icon, or by
list" button on the movie
activity, the system
er can only add or
thin any 2-minute
more details.
this task require that
nit tests. Use failing
ASUS Vivobook

README.md

Markdown Preview

AI Assistant

Error Responses:

Missing movie ID (400):

```
{
  "message": "Movie ID is required"
}
```

Invalid movie ID (400):

```
{
  "message": "Invalid movie ID format"
}
```

Movie not found in watchlist (404):

```
{
  "message": "Movie not found in watchlist"
}
```

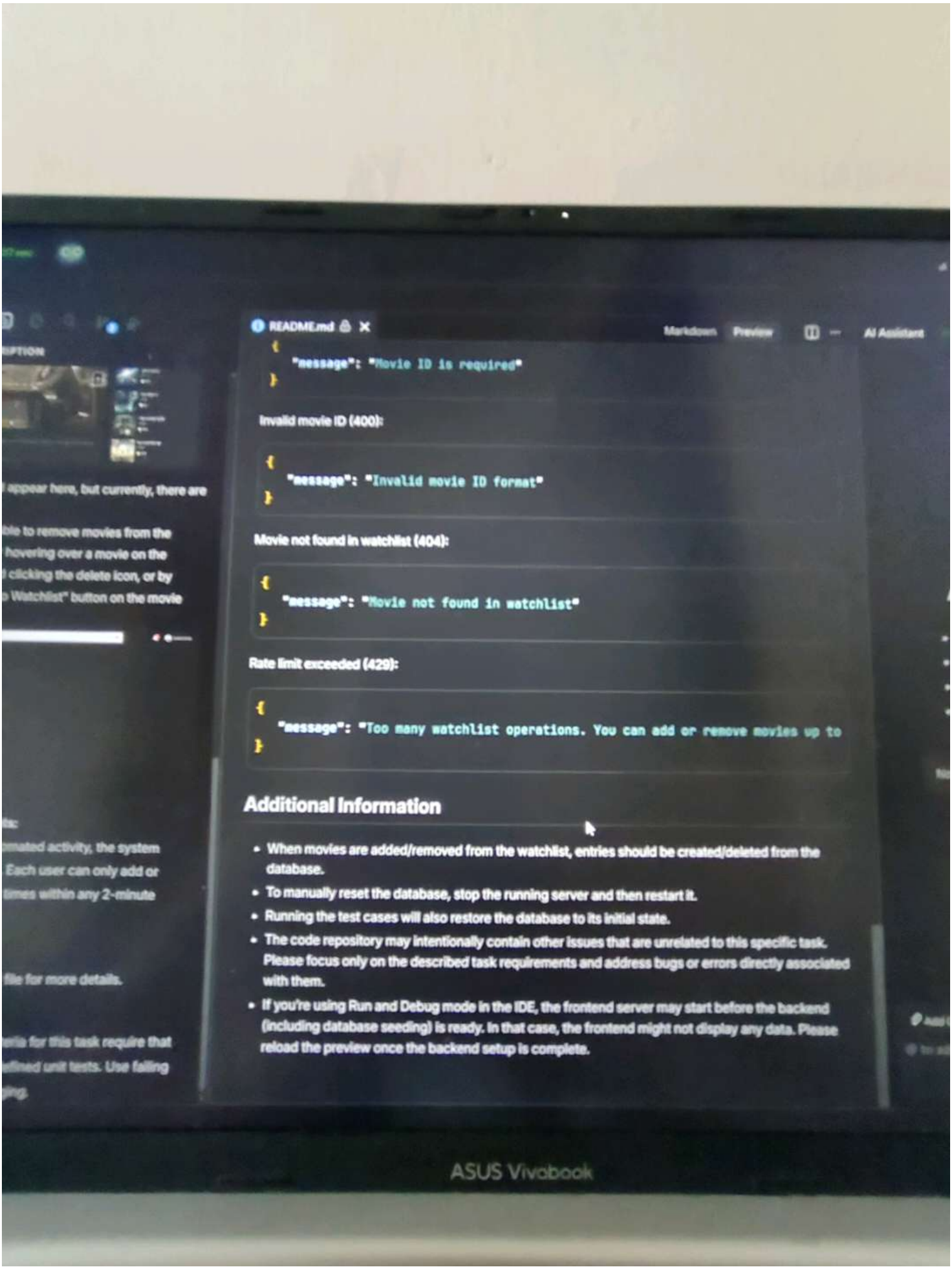
Rate limit exceeded (429):

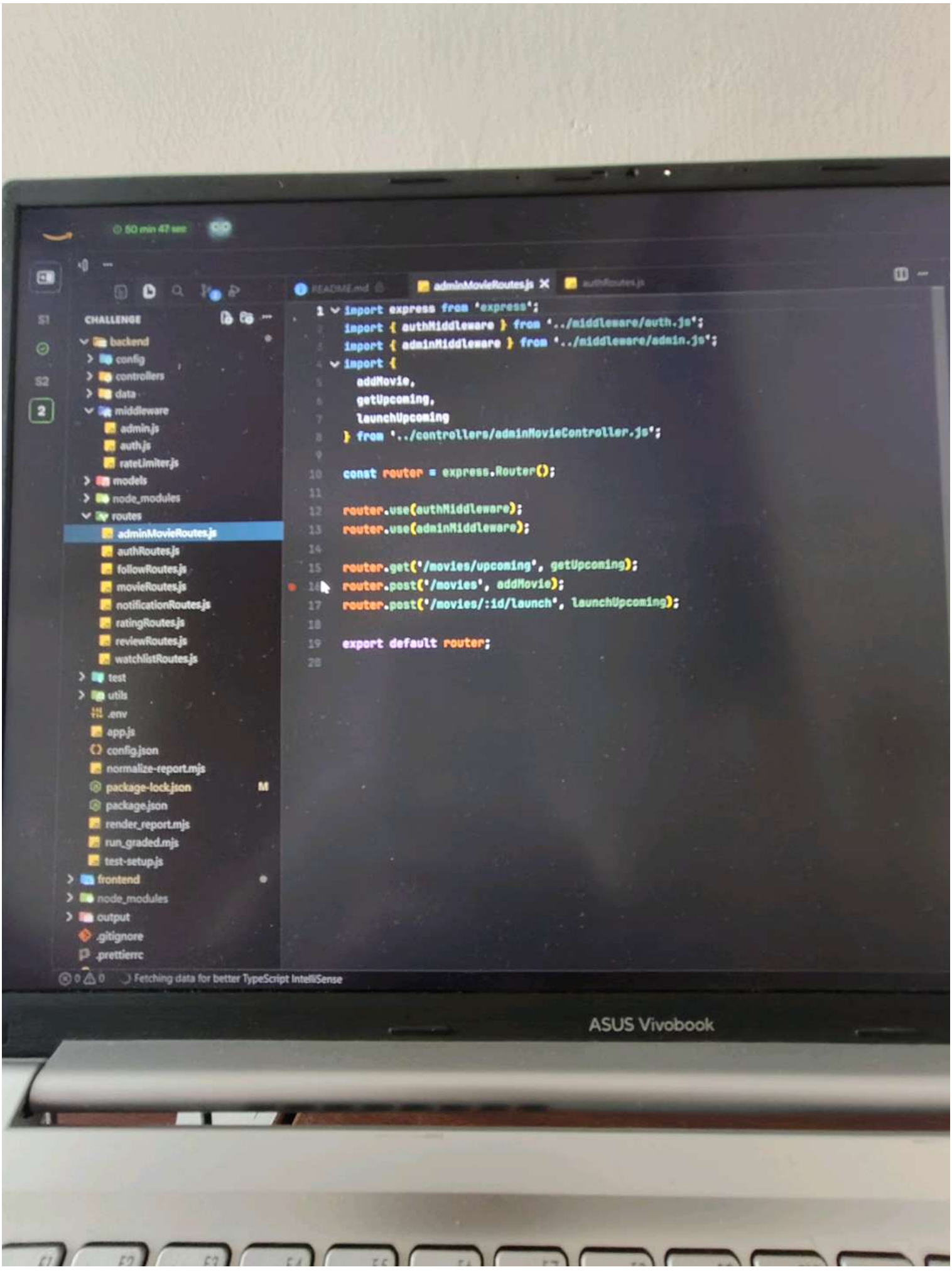
```
{
  "message": "Too many watchlist operations. You can add or remove movies up to
}
```

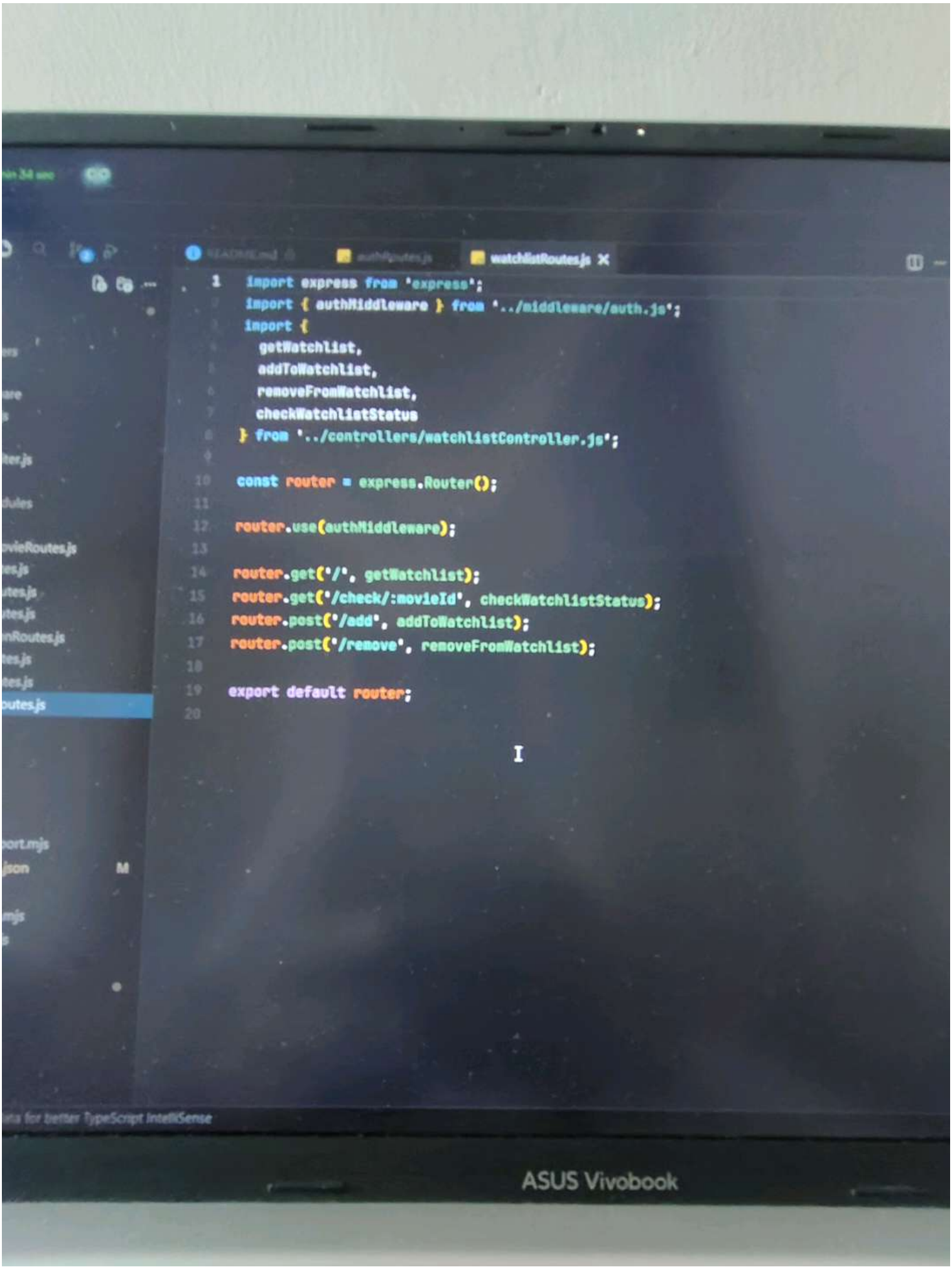
Additional Information

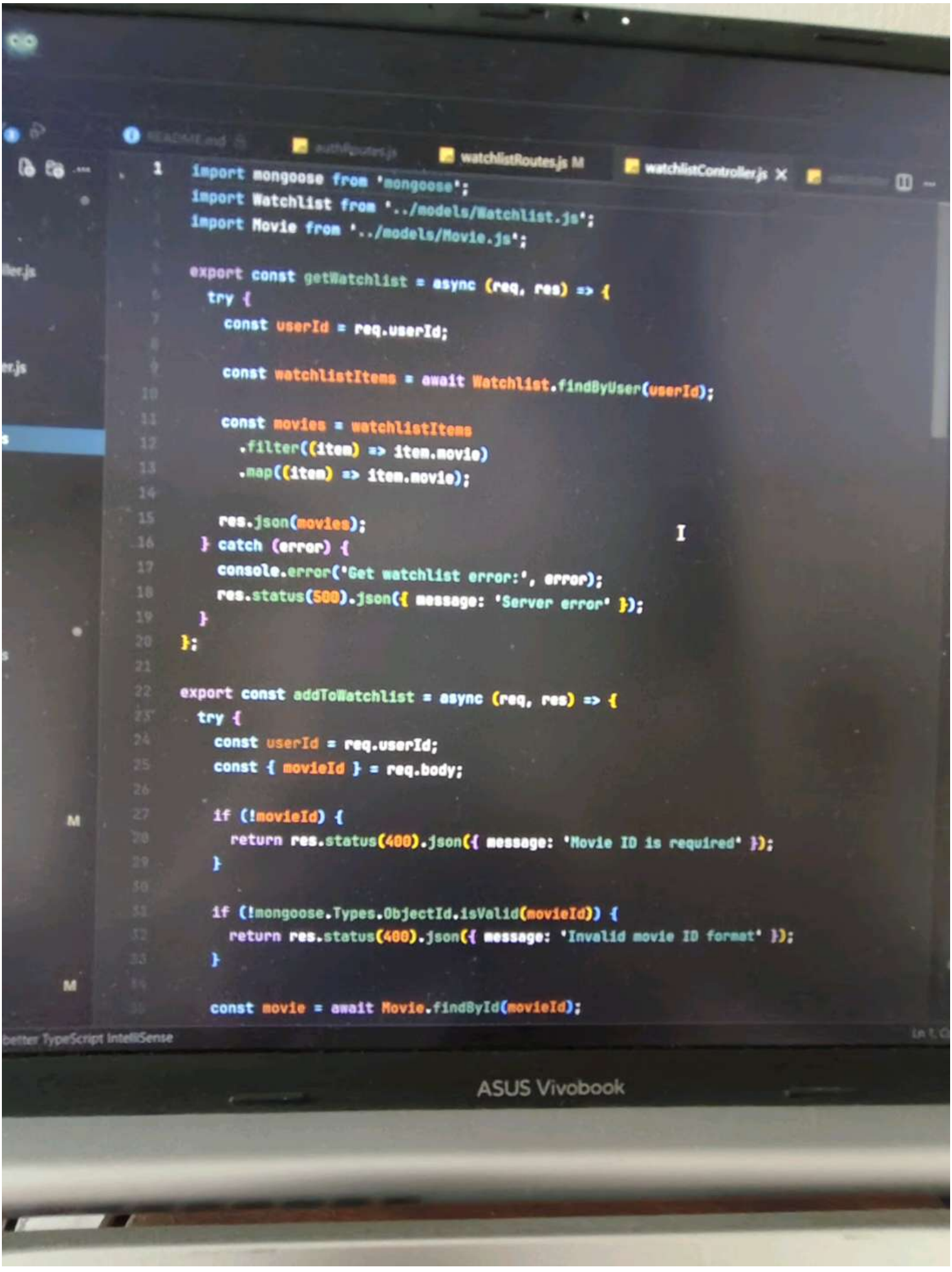
- When movies are added/removed from the watchlist, entries should be created/deleted from the database.
- To manually reset the database, stop the running server and then restart it.
- Running the test cases will also restore the database to its initial state.
- The code repository may intentionally contain other issues that are unrelated to this specific task. Please focus only on the described task requirements and address bugs or errors directly associated with them.

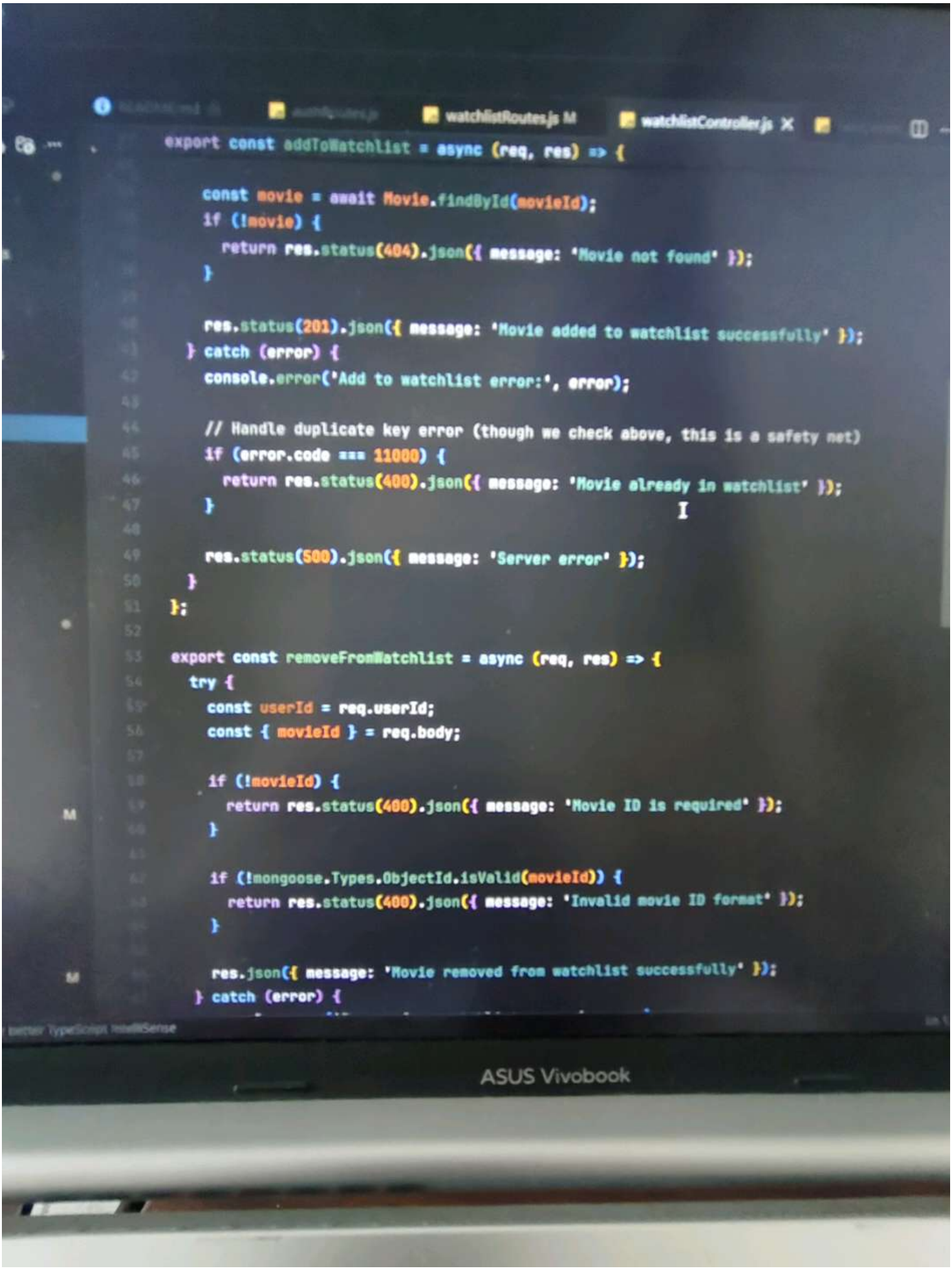
ASUS Vivobook

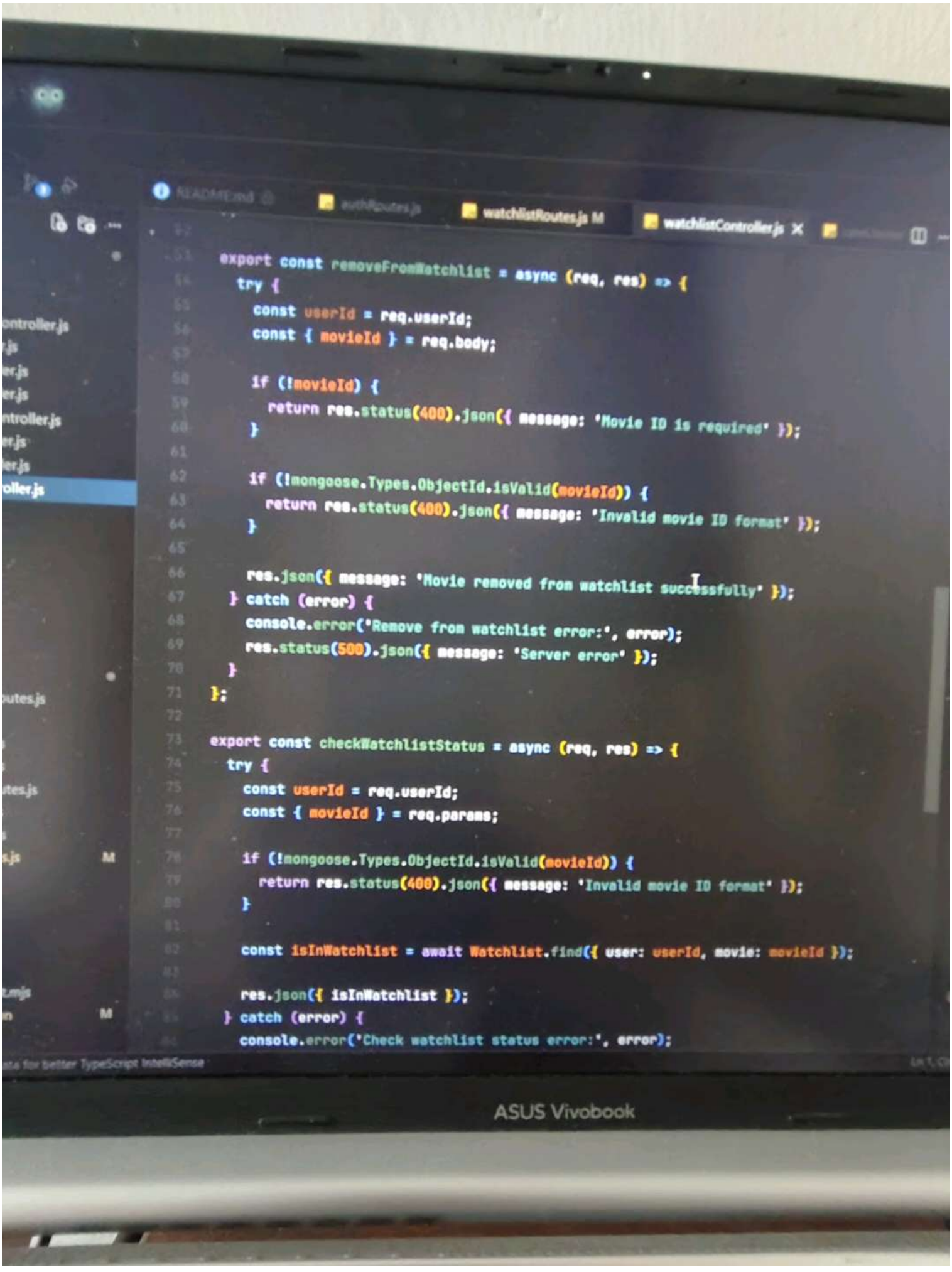

README.md
Markdown
Preview
AI Assistant
{
"message": "Movie ID is required"
}
Invalid movie ID (400):
{
"message": "Invalid movie ID format"
}
Movie not found in watchlist (404):
{
"message": "Movie not found in watchlist"
}
Rate limit exceeded (429):
{
"message": "Too many watchlist operations. You can add or remove movies up to
}
Additional Information
When movies are added/removed from the watchlist, entries should be created/deleted from the database.
To manually reset the database, stop the running server and then restart it.
Running the test cases will also restore the database to its initial state.
The code repository may intentionally contain other issues that are unrelated to this specific task. Please focus only on the described task requirements and address bugs or errors directly associated with them.
If you're using Run and Debug mode in the IDE, the frontend server may start before the backend (including database seeding) is ready. In that case, the frontend might not display any data. Please reload the preview once the backend setup is complete.
ASUS Vivobook

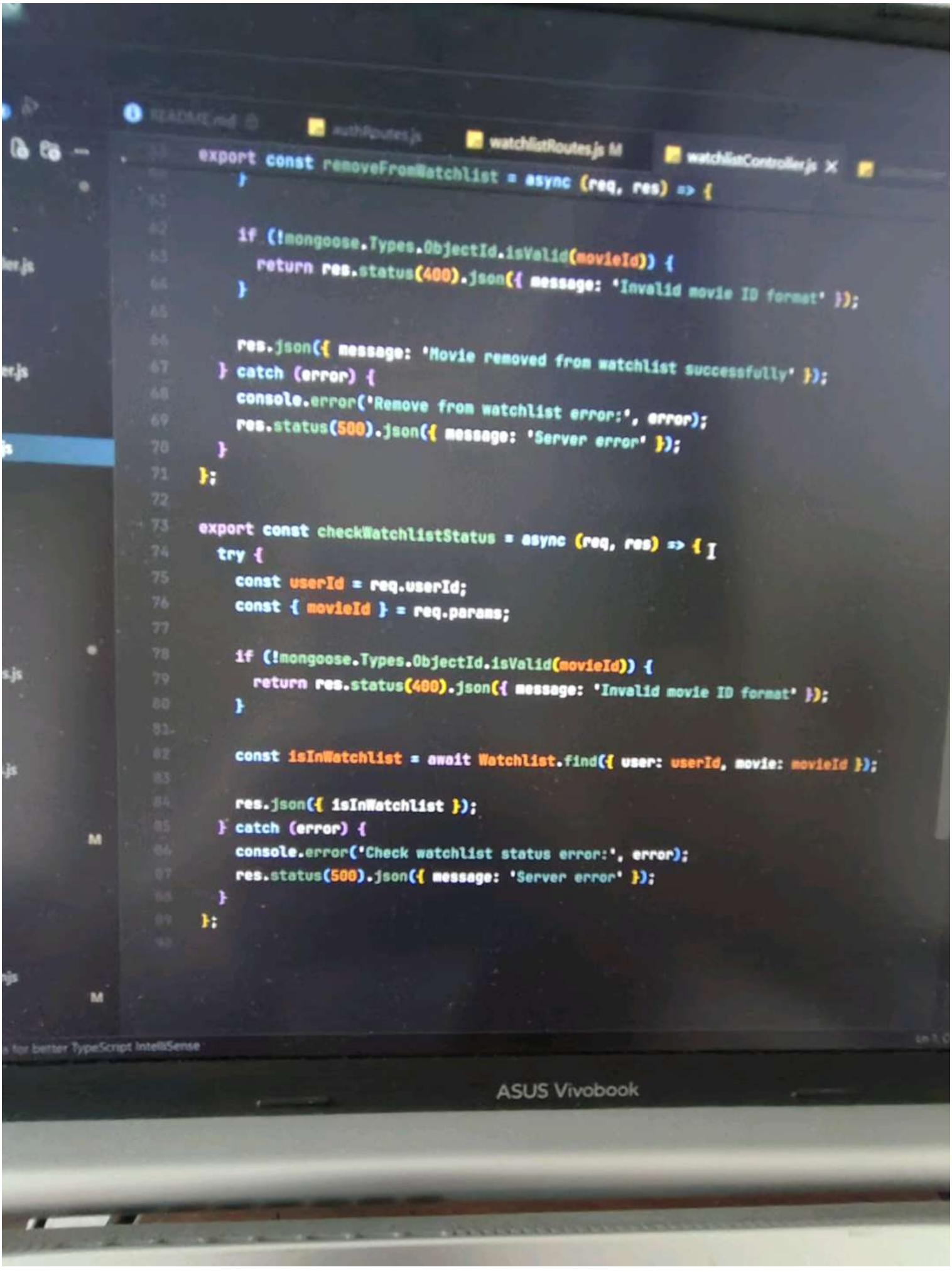

CHALLENGE
backend
config
controllers
data
middleware
admin.js
auth.js
rateLimiter.js
models
node_modules
routes
adminMovieRoutes.js
authRoutes.js
followRoutes.js
movieRoutes.js
notificationRoutes.js
ratingRoutes.js
reviewRoutes.js
watchlistRoutes.js
test
utils
.env
app.js
config.json
normalize-report.mjs
package-lock.json
package.json
render_report.mjs
run_graded.mjs
test-setup.js
frontend
node_modules
output
.gitignore
.prettierrc
README.md
adminMovieRoutes.js
import express from 'express';
import { authMiddleware } from '../middleware/auth.js';
import { adminMiddleware } from '../middleware/admin.js';
import {
addMovie,
getUpcoming,
launchUpcoming
} from '../controllers/adminMovieController.js';
const router = express.Router();
router.use(authMiddleware);
router.use(adminMiddleware);
router.get('/movies/upcoming', getUpcoming);
router.post('/movies', addMovie);
router.post('/movies/:id/launch', launchUpcoming);
export default router;
Fetching data for better TypeScript IntelliSense
ASUS Vivobook

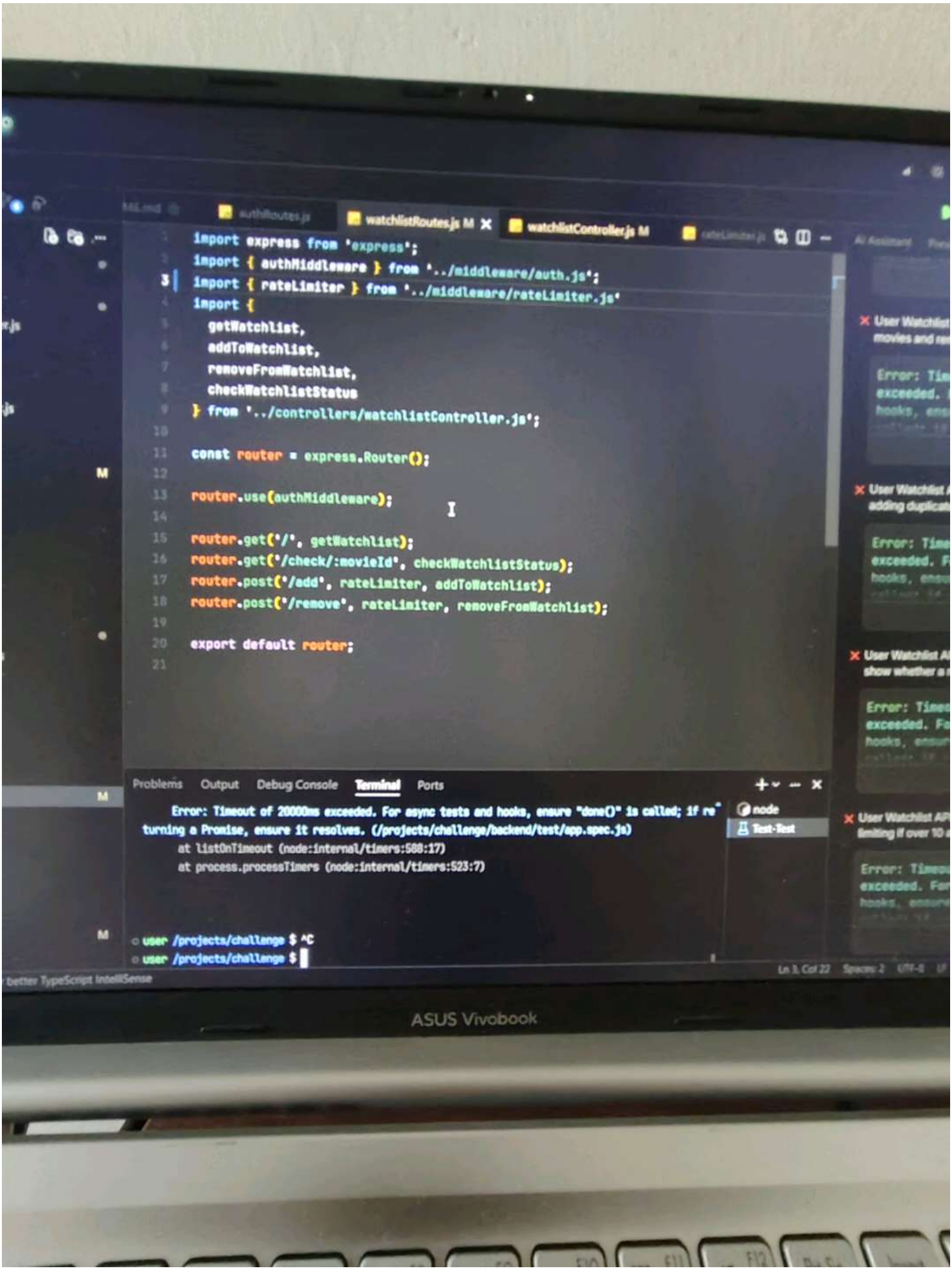

watchlistRoutes.js
import express from 'express';
import { authMiddleware } from '../middleware/auth.js';
import {
getWatchlist,
addToWatchlist,
removeFromWatchlist,
checkWatchlistStatus
} from '../controllers/watchlistController.js';
const router = express.Router();
router.use(authMiddleware);
router.get('/', getWatchlist);
router.get('/check/:movieId', checkWatchlistStatus);
router.post('/add', addToWatchlist);
router.post('/remove', removeFromWatchlist);
export default router;
ASUS Vivobook

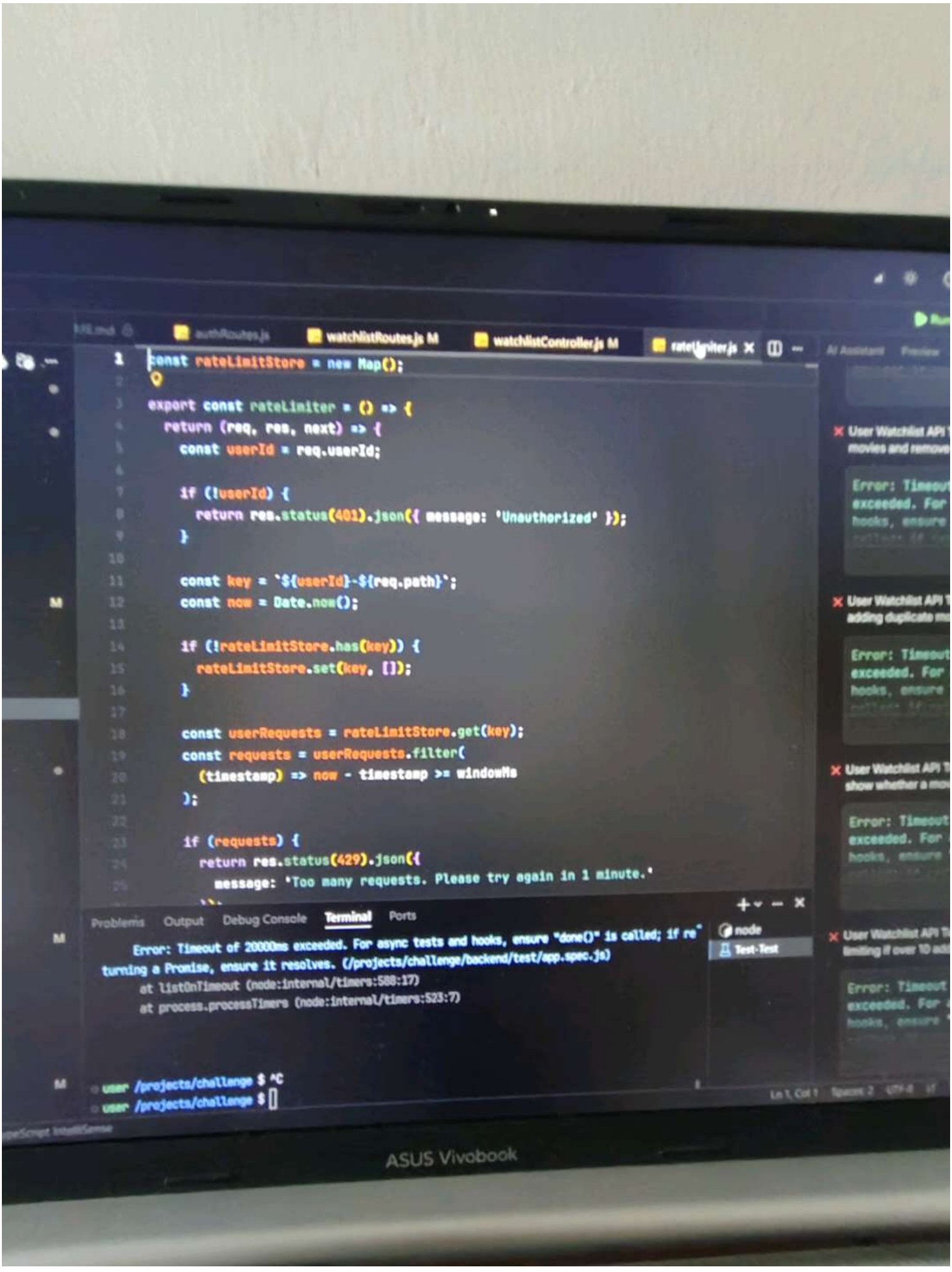

watchlistRoutes.js M
watchlistController.js
import mongoose from 'mongoose';
import Watchlist from '../models/Watchlist.js';
import Movie from '../models/Movie.js';
export const getWatchlist = async (req, res) => {
try {
const userId = req.userId;
const watchlistItems = await Watchlist.findByUser(userId);
const movies = watchlistItems
.filter((item) => item.movie)
.map((item) => item.movie);
res.json(movies);
} catch (error) {
console.error('Get watchlist error:', error);
res.status(500).json({ message: 'Server error' });
}
};
export const addToWatchlist = async (req, res) => {
try {
const userId = req.userId;
const { movieId } = req.body;
if (!movieId) {
return res.status(400).json({ message: 'Movie ID is required' });
}
if (!mongoose.Types.ObjectId.isValid(movieId)) {
return res.status(400).json({ message: 'Invalid movie ID format' });
}
const movie = await Movie.findById(movieId);
better TypeScript IntelliSense
ASUS Vivobook

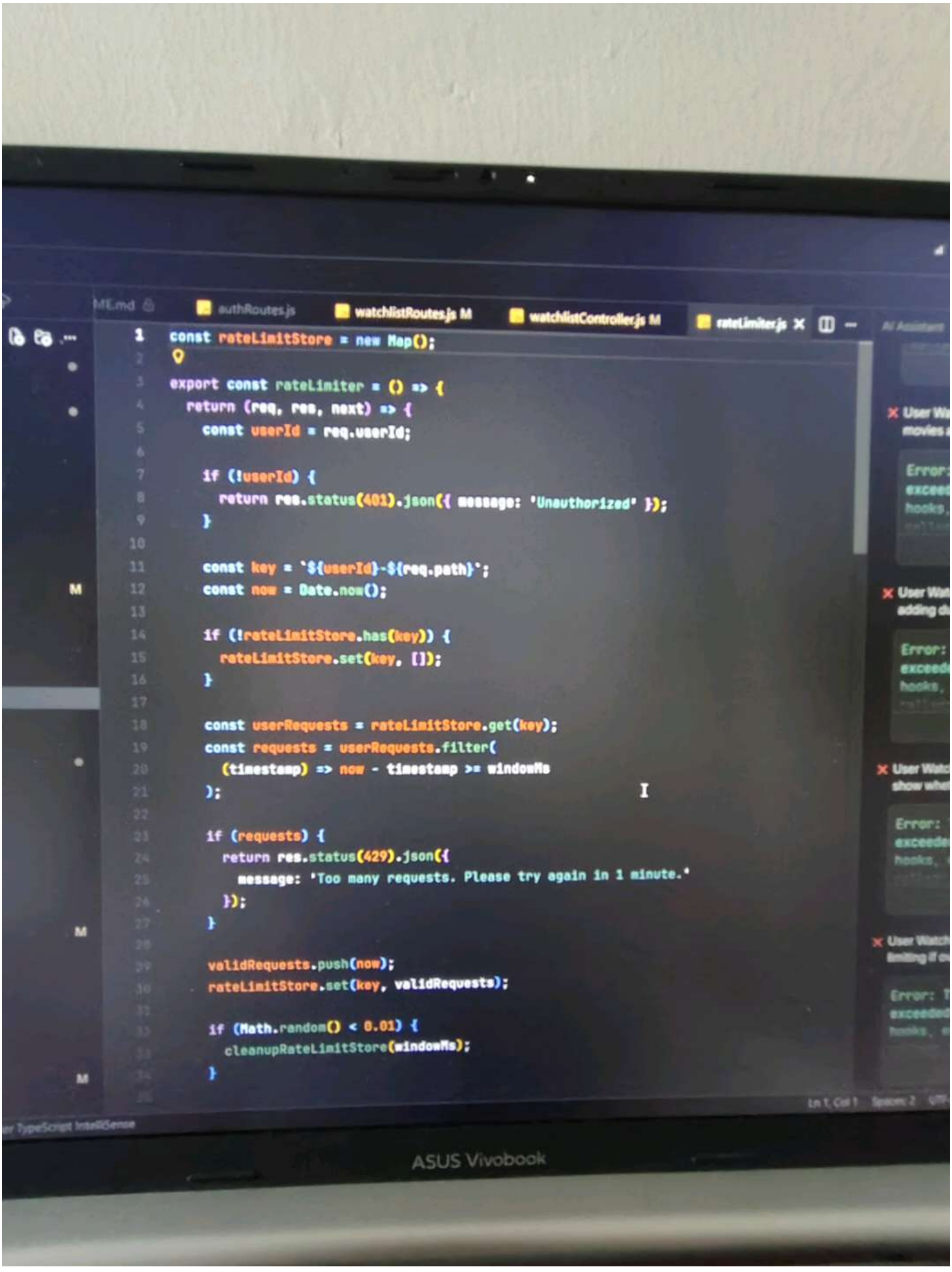

watchlistRoutes.js M
watchlistController.js
export const addToWatchlist = async (req, res) => {
const movie = await Movie.findById(movieId);
if (!movie) {
return res.status(404).json({ message: 'Movie not found' });
}
res.status(201).json({ message: 'Movie added to watchlist successfully' });
} catch (error) {
console.error('Add to watchlist error:', error);
// Handle duplicate key error (though we check above, this is a safety net)
if (error.code === 11000) {
return res.status(400).json({ message: 'Movie already in watchlist' });
}
res.status(500).json({ message: 'Server error' });
}
};
export const removeFromWatchlist = async (req, res) => {
try {
const userId = req.userId;
const { movieId } = req.body;
if (!movieId) {
return res.status(400).json({ message: 'Movie ID is required' });
}
if (!mongoose.Types.ObjectId.isValid(movieId)) {
return res.status(400).json({ message: 'Invalid movie ID format' });
}
res.json({ message: 'Movie removed from watchlist successfully' });
} catch (error) {
ASUS Vivobook

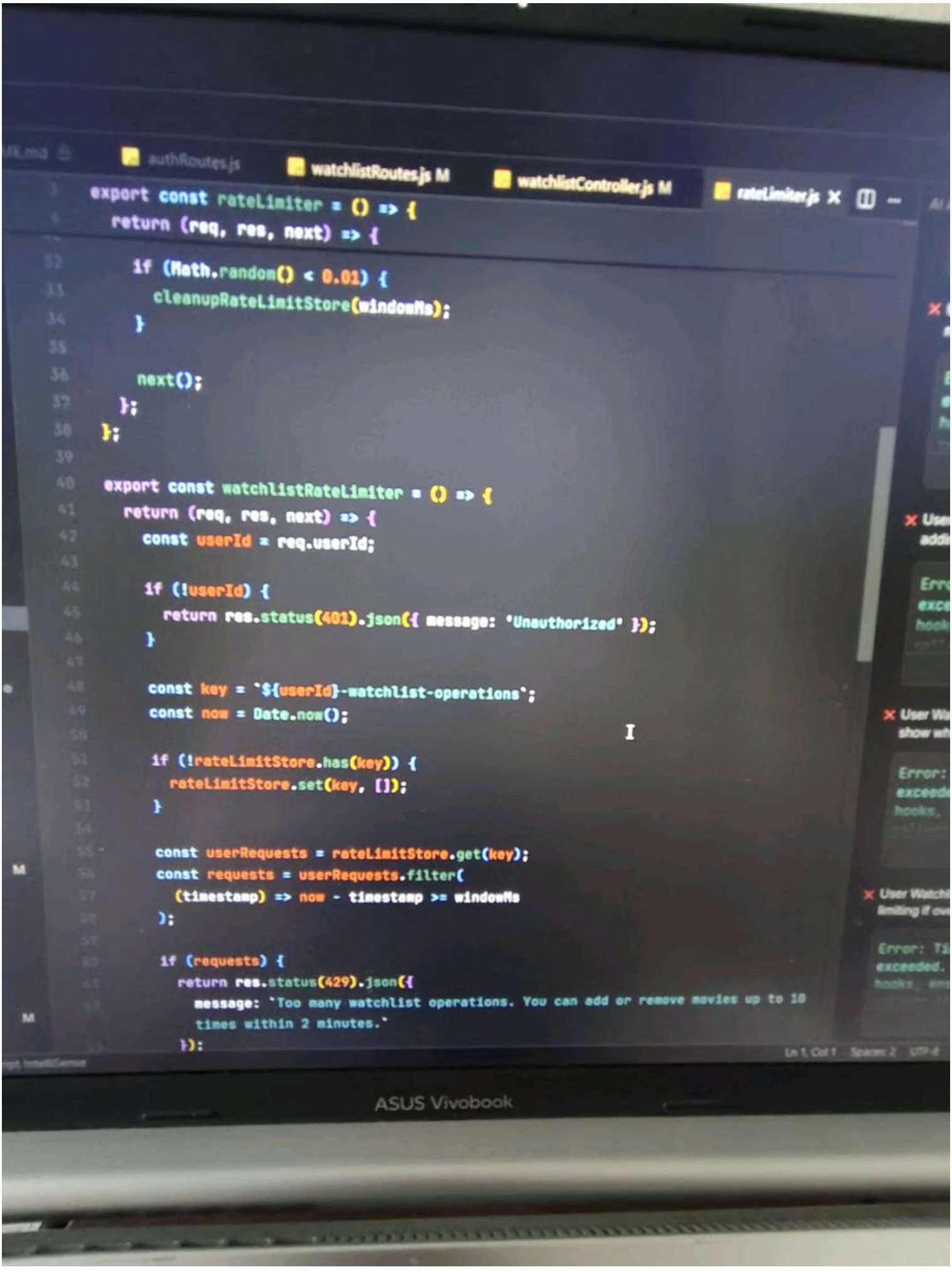

README.md
authRoutes.js
watchlistRoutes.js M
watchlistController.js
export const removeFromWatchlist = async (req, res) => {
try {
const userId = req.userId;
const { movieId } = req.body;
if (!movieId) {
return res.status(400).json({ message: 'Movie ID is required' });
}
if (!mongoose.Types.ObjectId.isValid(movieId)) {
return res.status(400).json({ message: 'Invalid movie ID format' });
}
res.json({ message: 'Movie removed from watchlist successfully' });
} catch (error) {
console.error('Remove from watchlist error:', error);
res.status(500).json({ message: 'Server error' });
}
};
export const checkWatchlistStatus = async (req, res) => {
try {
const userId = req.userId;
const { movieId } = req.params;
if (!mongoose.Types.ObjectId.isValid(movieId)) {
return res.status(400).json({ message: 'Invalid movie ID format' });
}
const isInWatchlist = await Watchlist.find({ user: userId, movie: movieId });
res.json({ isInWatchlist });
} catch (error) {
console.error('Check watchlist status error:', error);
ASUS Vivobook

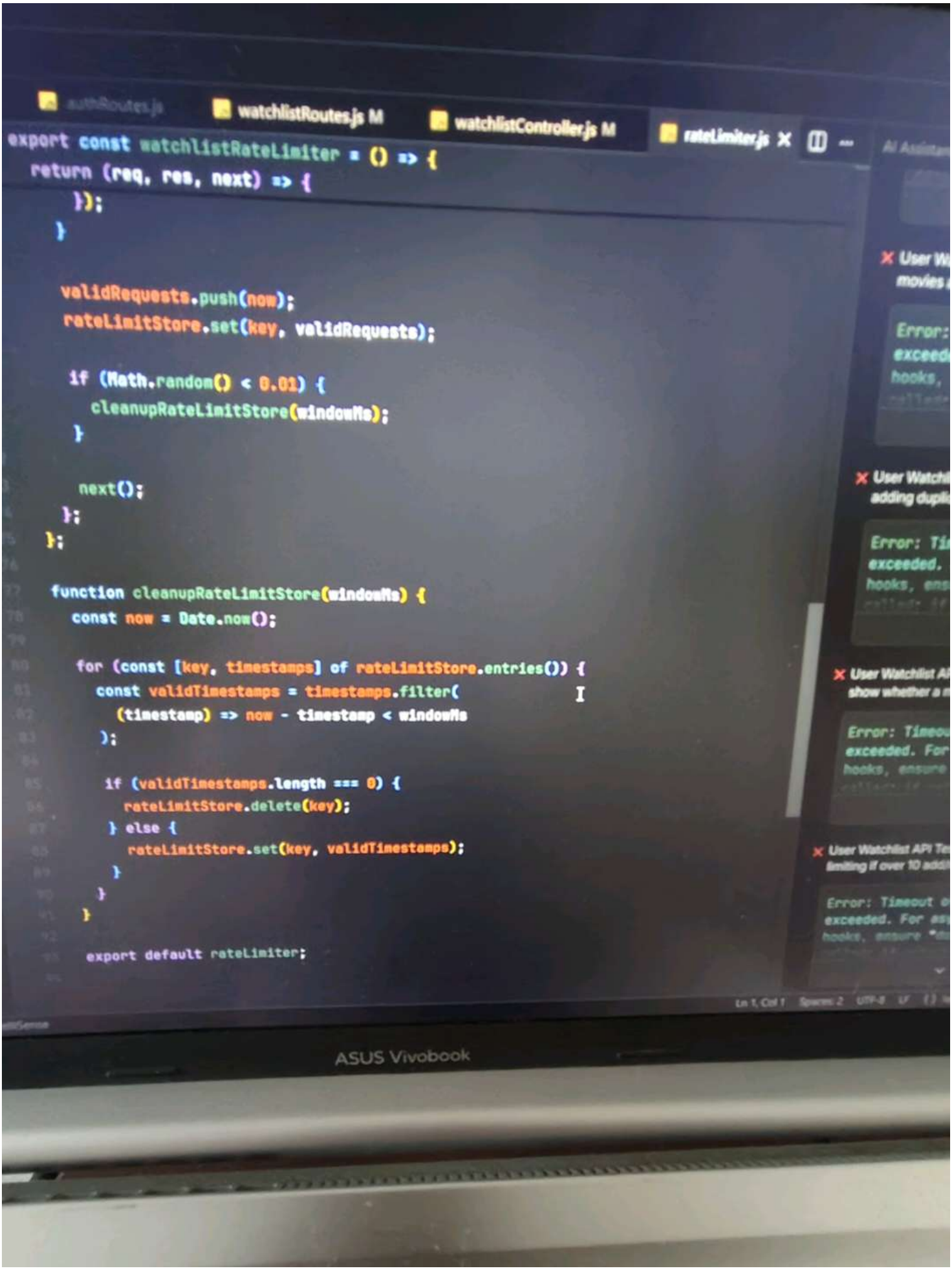

watchlistRoutes.js M
watchlistController.js
export const removeFromWatchlist = async (req, res) => {
if (!mongoose.Types.ObjectId.isValid(movieId)) {
return res.status(400).json({ message: 'Invalid movie ID format' });
res.json({ message: 'Movie removed from watchlist successfully' });
} catch (error) {
console.error('Remove from watchlist error:', error);
res.status(500).json({ message: 'Server error' });
};
export const checkWatchlistStatus = async (req, res) => {
try {
const userId = req.userId;
const { movieId } = req.params;
if (!mongoose.Types.ObjectId.isValid(movieId)) {
return res.status(400).json({ message: 'Invalid movie ID format' });
const isInWatchlist = await Watchlist.find({ user: userId, movie: movieId });
res.json({ isInWatchlist });
} catch (error) {
console.error('Check watchlist status error:', error);
res.status(500).json({ message: 'Server error' });
};
ASUS Vivobook

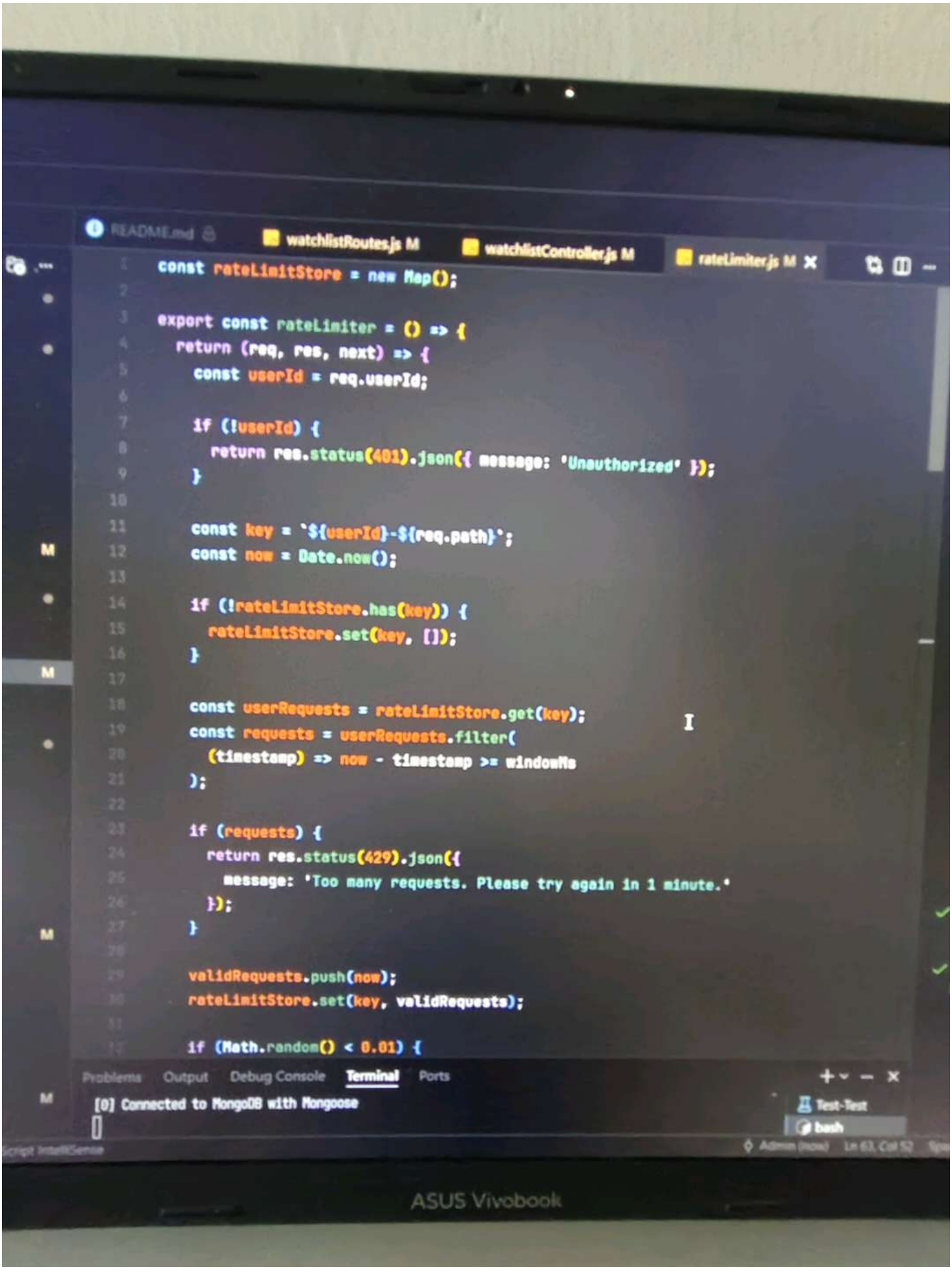

authRoutes.js
watchlistRoutes.js M
watchlistController.js M
import express from 'express';
import { authMiddleware } from '../middleware/auth.js';
import { rateLimiter } from '../middleware/rateLimiter.js'
import {
getWatchlist,
addToWatchlist,
removeFromWatchlist,
checkWatchlistStatus
} from '../controllers/watchlistController.js';
const router = express.Router();
router.use(authMiddleware);
router.get('/', getWatchlist);
router.get('/check/:movieId', checkWatchlistStatus);
router.post('/add', rateLimiter, addToWatchlist);
router.post('/remove', rateLimiter, removeFromWatchlist);
export default router;
Problems
Output
Debug Console
Terminal
Ports
Error: Timeout of 20000ms exceeded. For async tests and hooks, ensure "done()" is called; if re
turning a Promise, ensure it resolves. (/projects/challenge/backend/test/app.spec.js)
at listOnTimeout (node:internal/timers:588:17)
at process.processTimers (node:internal/timers:523:7)
node
Test-Test
user /projects/challenge $ ^C
user /projects/challenge $
ASUS Vivobook

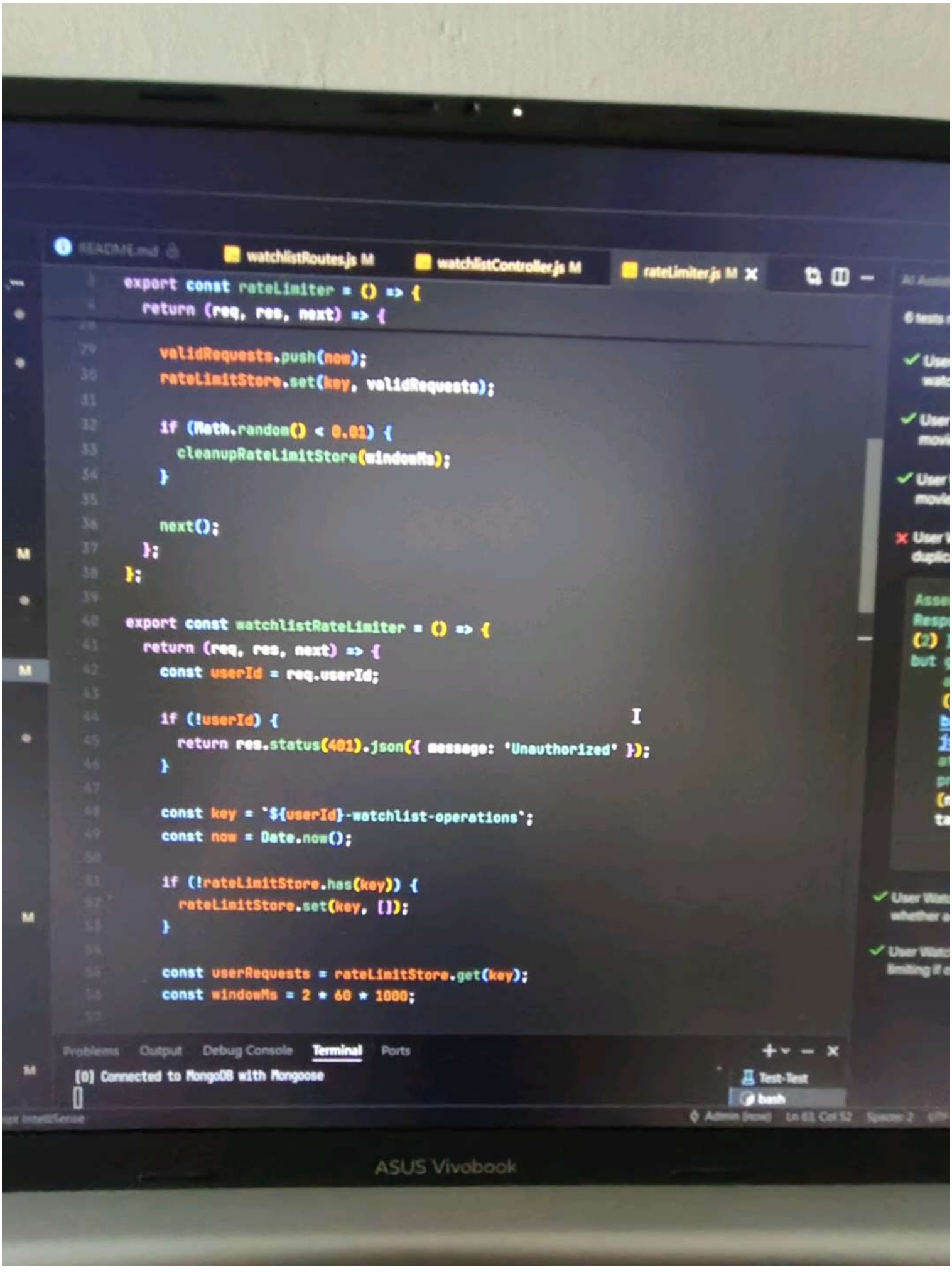

authRoutes.js
watchlistRoutes.js M
watchlistController.js M
rateLimiter.js
const rateLimitStore = new Map();
export const rateLimiter = () => {
return (req, res, next) => {
const userId = req.userId;
if (!userId) {
return res.status(401).json({ message: 'Unauthorized' });
}
const key = `${userId}-${req.path}`;
const now = Date.now();
if (!rateLimitStore.has(key)) {
rateLimitStore.set(key, []);
}
const userRequests = rateLimitStore.get(key);
const requests = userRequests.filter(
(timestamp) => now - timestamp >= windowMs
);
if (requests) {
return res.status(429).json({
message: 'Too many requests. Please try again in 1 minute.'
Problems
Output
Debug Console
Terminal
Ports
node
Test-Test
Error: Timeout of 20000ms exceeded. For async tests and hooks, ensure "done()" is called; if re
turning a Promise, ensure it resolves. (/projects/challenge/backend/test/app.spec.js)
at listOnTimeout (node:internal/timers:580:17)
at process.processTimers (node:internal/timers:523:7)
user /projects/challenge $ ^C
user /projects/challenge $
AI Assistant
Preview
User Watchlist API
movies and remove
Error: Timeout
exceeded. For
hooks, ensure
User Watchlist API
adding duplicate
Error: Timeout
exceeded. For
hooks, ensure
User Watchlist API
show whether a
Error: Timeout
exceeded. For
hooks, ensure
User Watchlist API
limiting if over 10
Error: Timeout
exceeded. For
hooks, ensure
ASUS Vivobook

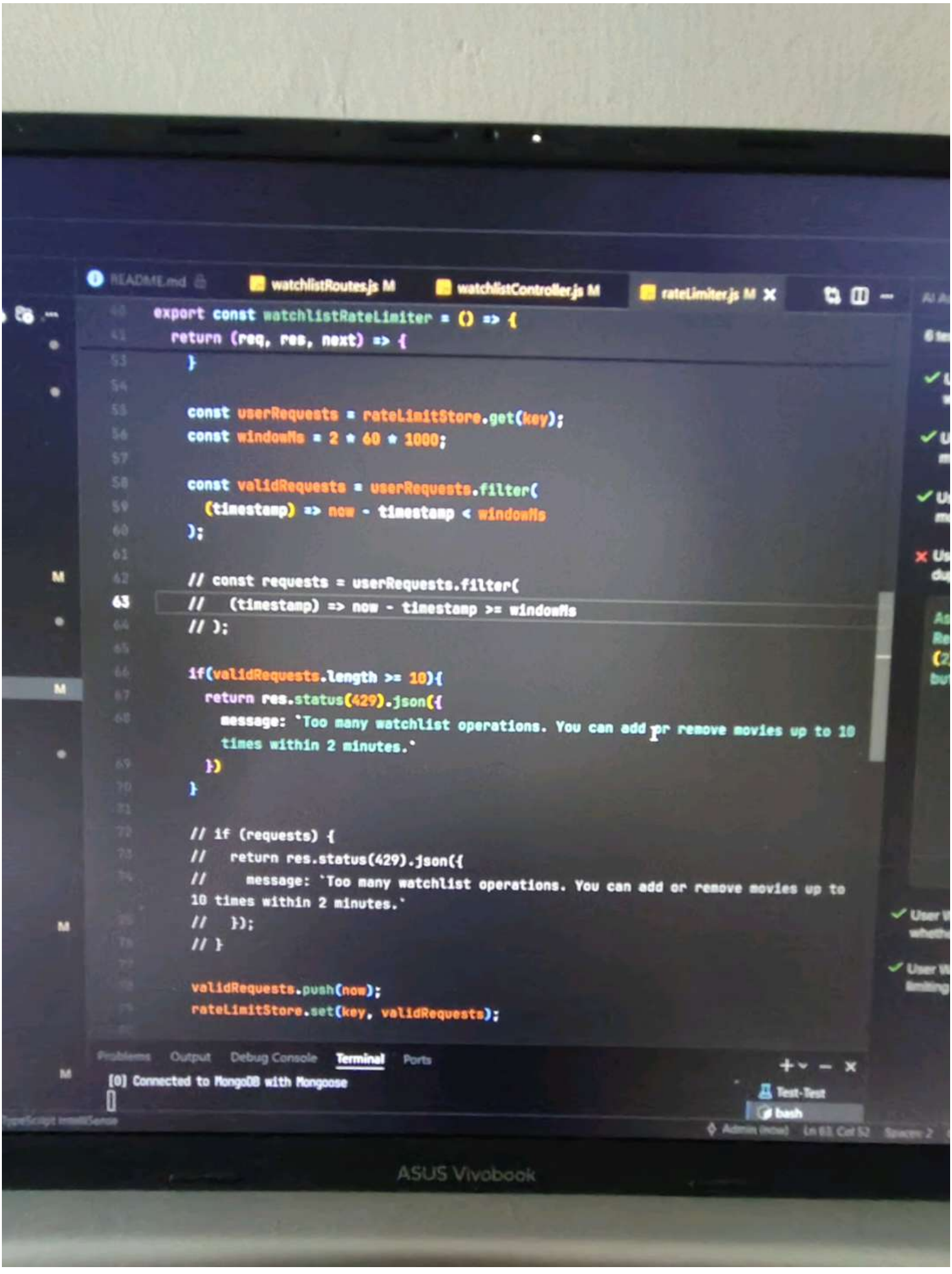

authRoutes.js
watchlistRoutes.js M
watchlistController.js M
rateLimiter.js
const rateLimitStore = new Map();
export const rateLimiter = () => {
return (req, res, next) => {
const userId = req.userId;
if (!userId) {
return res.status(401).json({ message: 'Unauthorized' });
}
const key = `${userId}-${req.path}`;
const now = Date.now();
if (!rateLimitStore.has(key)) {
rateLimitStore.set(key, []);
}
const userRequests = rateLimitStore.get(key);
const requests = userRequests.filter(
(timestamp) => now - timestamp >= windowMs
);
if (requests) {
return res.status(429).json({
message: 'Too many requests. Please try again in 1 minute.'
});
}
validRequests.push(now);
rateLimitStore.set(key, validRequests);
if (Math.random() < 0.01) {
cleanupRateLimitStore(windowMs);
}
ASUS Vivobook

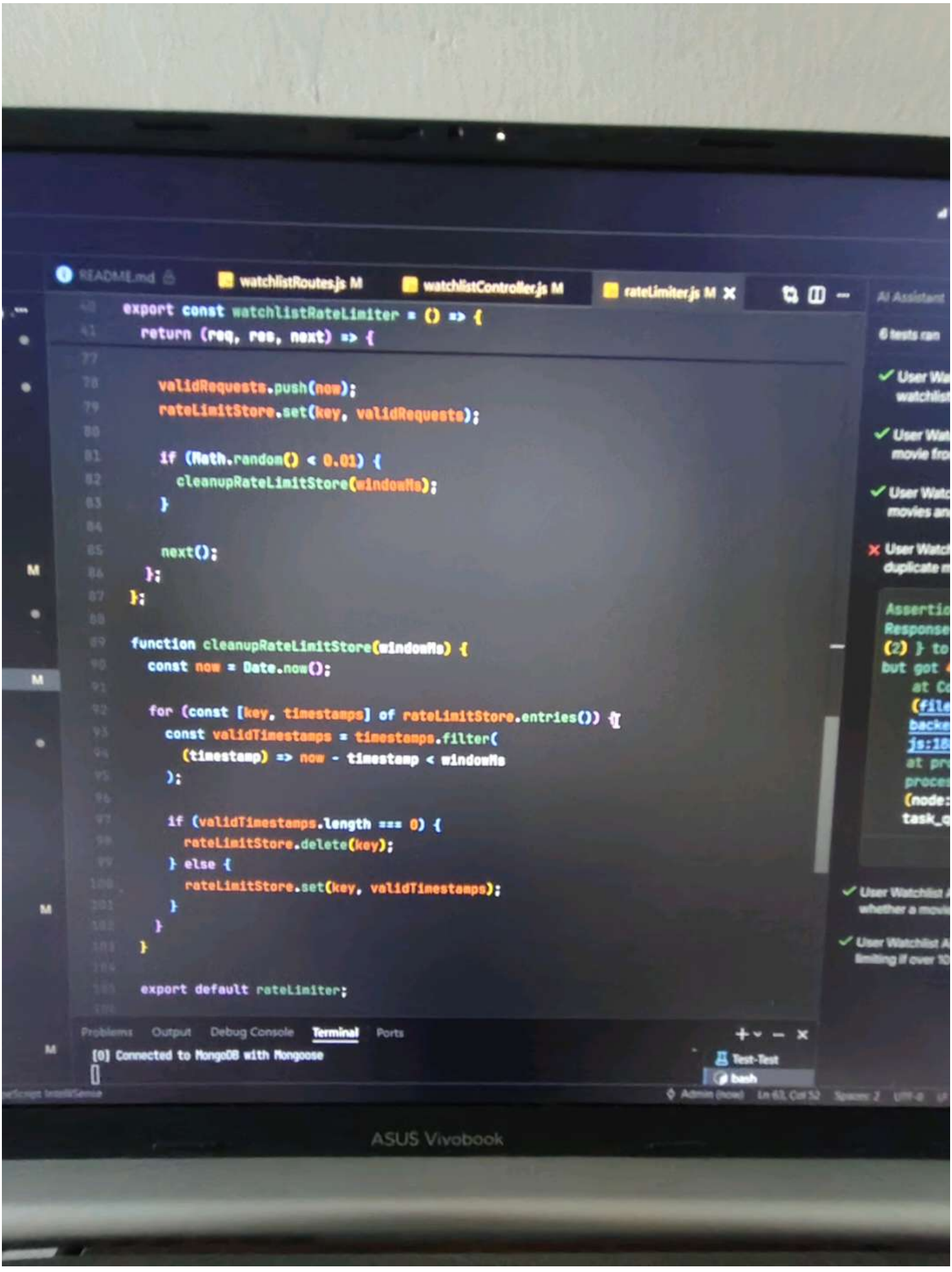

authRoutes.js
watchlistRoutes.js M
watchlistController.js M
rateLimiter.js
export const rateLimiter = () => {
return (req, res, next) => {
if (Math.random() < 0.01) {
cleanupRateLimitStore(windowMs);
}
next();
};
};
export const watchlistRateLimiter = () => {
return (req, res, next) => {
const userId = req.userId;
if (!userId) {
return res.status(401).json({ message: 'Unauthorized' });
}
const key = `${userId}-watchlist-operations`;
const now = Date.now();
if (!rateLimitStore.has(key)) {
rateLimitStore.set(key, []);
}
const userRequests = rateLimitStore.get(key);
const requests = userRequests.filter(
(timestamp) => now - timestamp >= windowMs
);
if (requests) {
return res.status(429).json({
message: `Too many watchlist operations. You can add or remove movies up to 10
times within 2 minutes.`
});
ASUS Vivobook

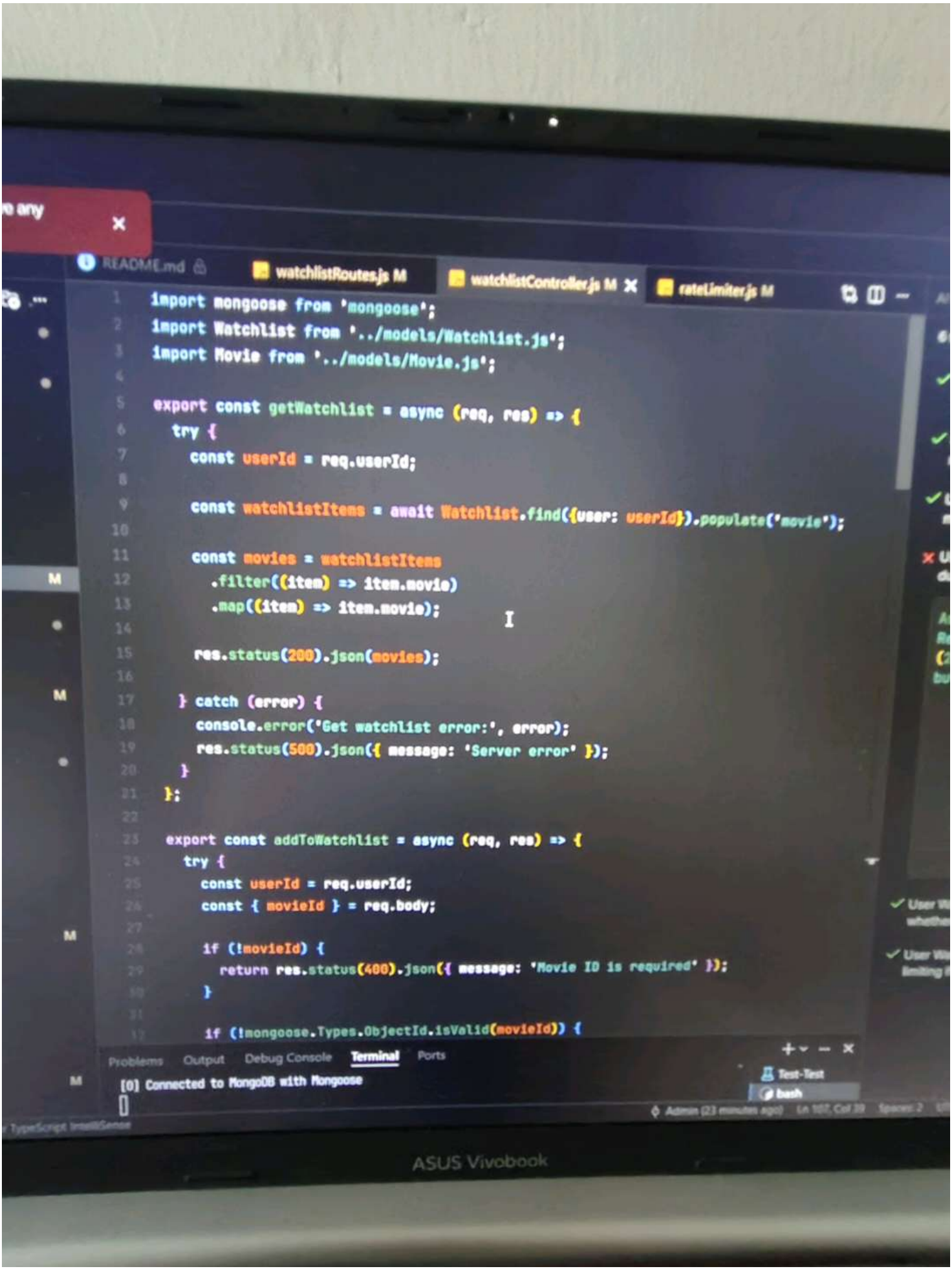

authRoutes.js
watchlistRoutes.js M
watchlistController.js M
rateLimiter.js
export const watchlistRateLimiter = () => {
return (req, res, next) => {
});
}
validRequests.push(now);
rateLimitStore.set(key, validRequests);
if (Math.random() < 0.01) {
cleanupRateLimitStore(windowMs);
}
next();
};
};
function cleanupRateLimitStore(windowMs) {
const now = Date.now();
for (const [key, timestamps] of rateLimitStore.entries()) {
const validTimestamps = timestamps.filter(
(timestamp) => now - timestamp < windowMs
);
if (validTimestamps.length === 0) {
rateLimitStore.delete(key);
} else {
rateLimitStore.set(key, validTimestamps);
}
}
}
export default rateLimiter;
ASUS Vivobook

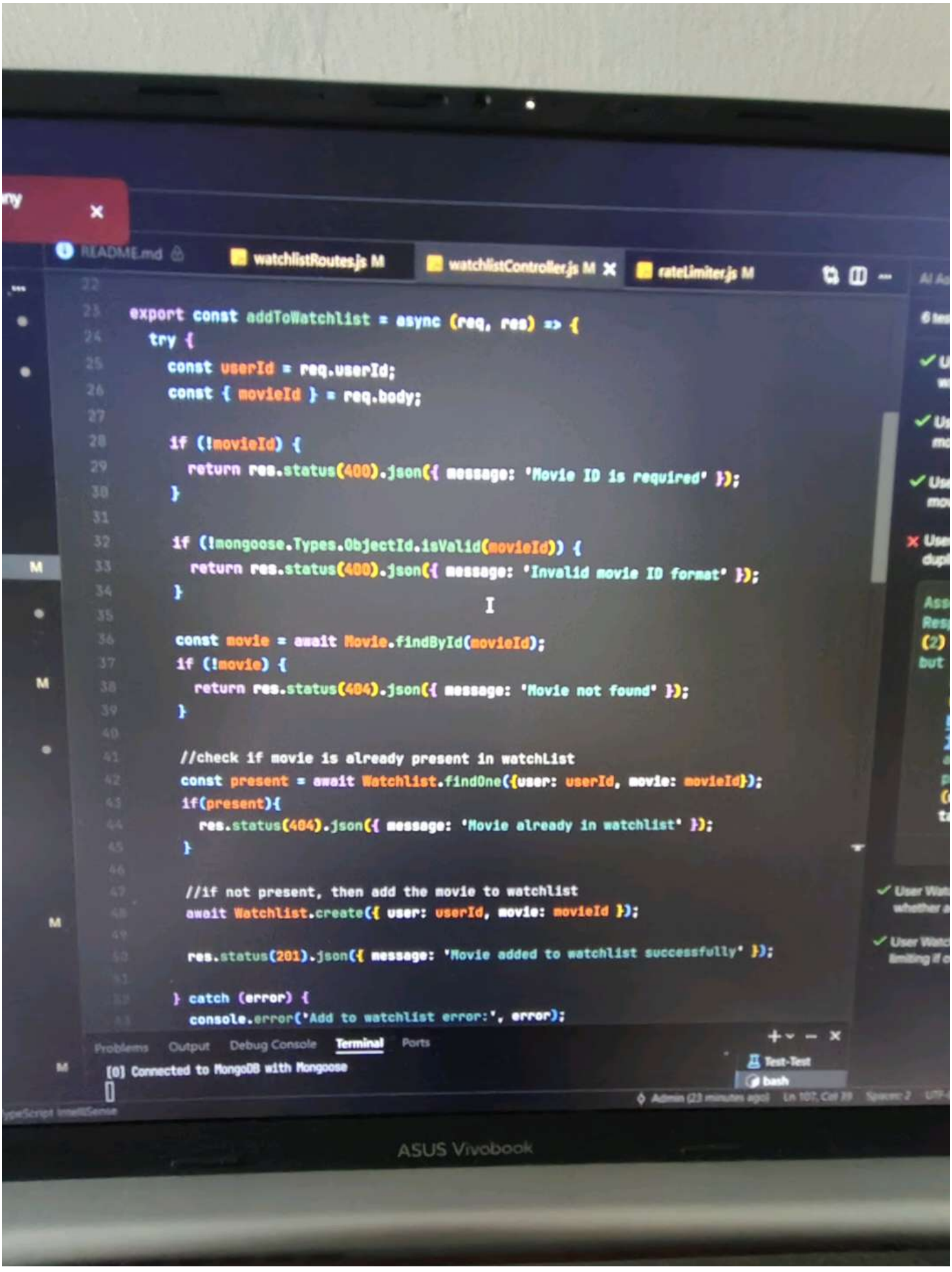

README.md
watchlistRoutes.js M
watchlistController.js M
rateLimiter.js M
const rateLimitStore = new Map();
export const rateLimiter = () => {
return (req, res, next) => {
const userId = req.userId;
if (!userId) {
return res.status(401).json({ message: 'Unauthorized' });
}
const key = `${userId}-${req.path}`;
const now = Date.now();
if (!rateLimitStore.has(key)) {
rateLimitStore.set(key, []);
}
const userRequests = rateLimitStore.get(key);
const requests = userRequests.filter(
(timestamp) => now - timestamp >= windowMs
);
if (requests) {
return res.status(429).json({
message: 'Too many requests. Please try again in 1 minute.'
});
}
validRequests.push(now);
rateLimitStore.set(key, validRequests);
if (Math.random() < 0.01) {
Problems
Output
Debug Console
Terminal
Ports
[0] Connected to MongoDB with Mongoose
Test-Test
bash
ASUS Vivobook

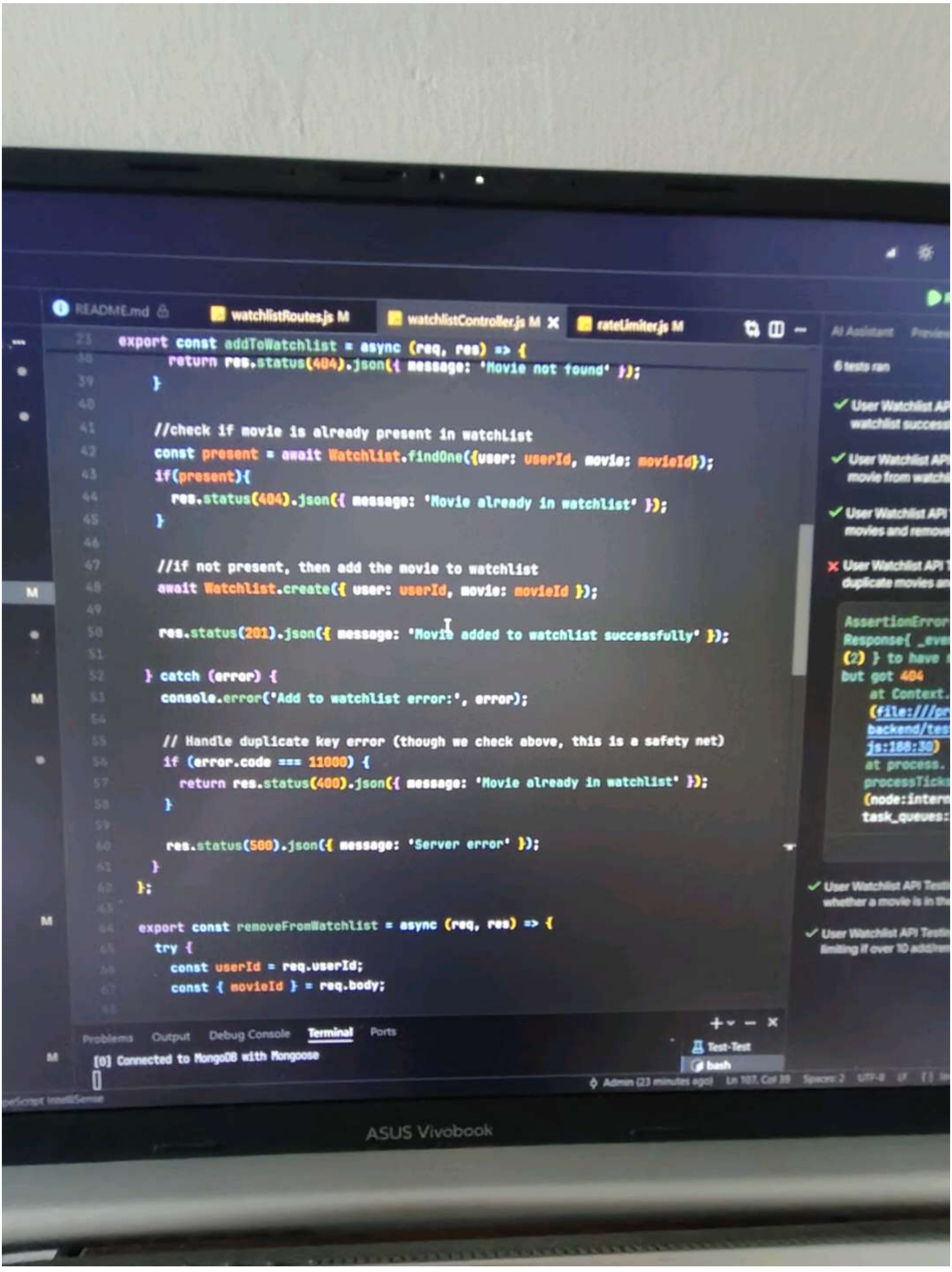

README.md
watchlistRoutes.js M
watchlistController.js M
rateLimiter.js M
export const rateLimiter = () => {
return (req, res, next) => {
validRequests.push(now);
rateLimitStore.set(key, validRequests);
if (Math.random() < 0.01) {
cleanupRateLimitStore(windowMs);
}
next();
};
};
export const watchlistRateLimiter = () => {
return (req, res, next) => {
const userId = req.userId;
if (!userId) {
return res.status(401).json({ message: 'Unauthorized' });
}
const key = `${userId}-watchlist-operations`;
const now = Date.now();
if (!rateLimitStore.has(key)) {
rateLimitStore.set(key, []);
}
const userRequests = rateLimitStore.get(key);
const windowMs = 2 * 60 * 1000;
Problems
Output
Debug Console
Terminal
Ports
[0] Connected to MongoDB with Mongoose
Test-Test
bash
ASUS Vivobook

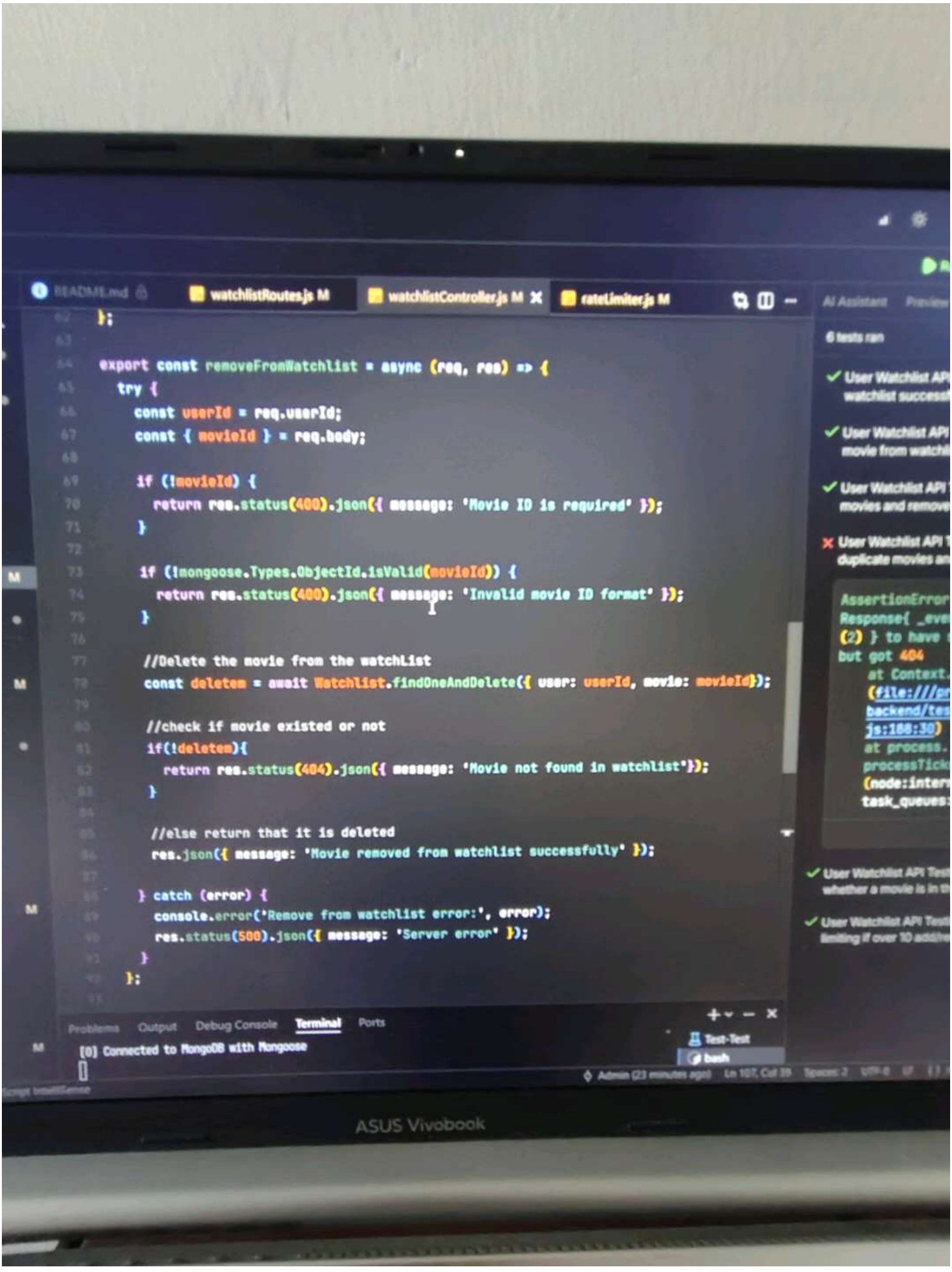

README.md
watchlistRoutes.js M
watchlistController.js M
rateLimiter.js M
export const watchlistRateLimiter = () => {
return (req, res, next) => {
}
const userRequests = rateLimitStore.get(key);
const windowMs = 2 * 60 * 1000;
const validRequests = userRequests.filter(
(timestamp) => now - timestamp < windowMs
);
// const requests = userRequests.filter(
// (timestamp) => now - timestamp >= windowMs
//);
if(validRequests.length >= 10){
return res.status(429).json({
message: `Too many watchlist operations. You can add or remove movies up to 10 times within 2 minutes.`
})
}
// if (requests) {
// return res.status(429).json({
// message: `Too many watchlist operations. You can add or remove movies up to 10 times within 2 minutes.`
// });
// }
validRequests.push(now);
rateLimitStore.set(key, validRequests);
Problems
Output
Debug Console
Terminal
Ports
[0] Connected to MongoDB with Mongoose
Test-Test
bash
ASUS Vivobook

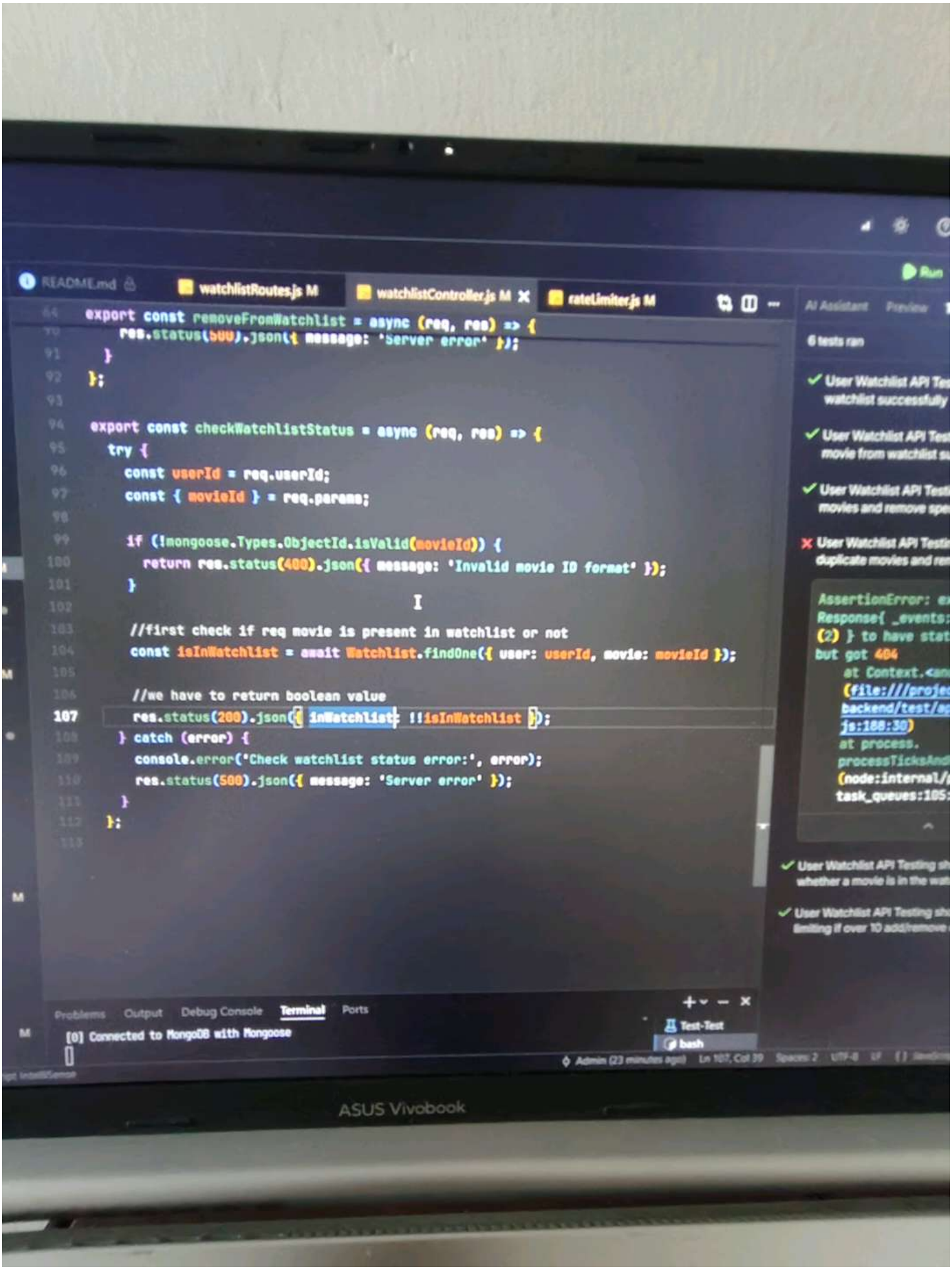

README.md
watchlistRoutes.js M
watchlistController.js M
rateLimiter.js M
export const watchlistRateLimiter = () => {
return (req, res, next) => {
validRequests.push(now);
rateLimitStore.set(key, validRequests);
if (Math.random() < 0.01) {
cleanupRateLimitStore(windowMs);
}
next();
};
};
function cleanupRateLimitStore(windowMs) {
const now = Date.now();
for (const [key, timestamps] of rateLimitStore.entries()) {
const validTimestamps = timestamps.filter(
(timestamp) => now - timestamp < windowMs
);
if (validTimestamps.length === 0) {
rateLimitStore.delete(key);
} else {
rateLimitStore.set(key, validTimestamps);
}
}
}
export default rateLimiter;
Problems
Output
Debug Console
Terminal
Ports
[0] Connected to MongoDB with Mongoose
Test-Test
bash
AI Assistant
6 tests ran
ASUS Vivobook

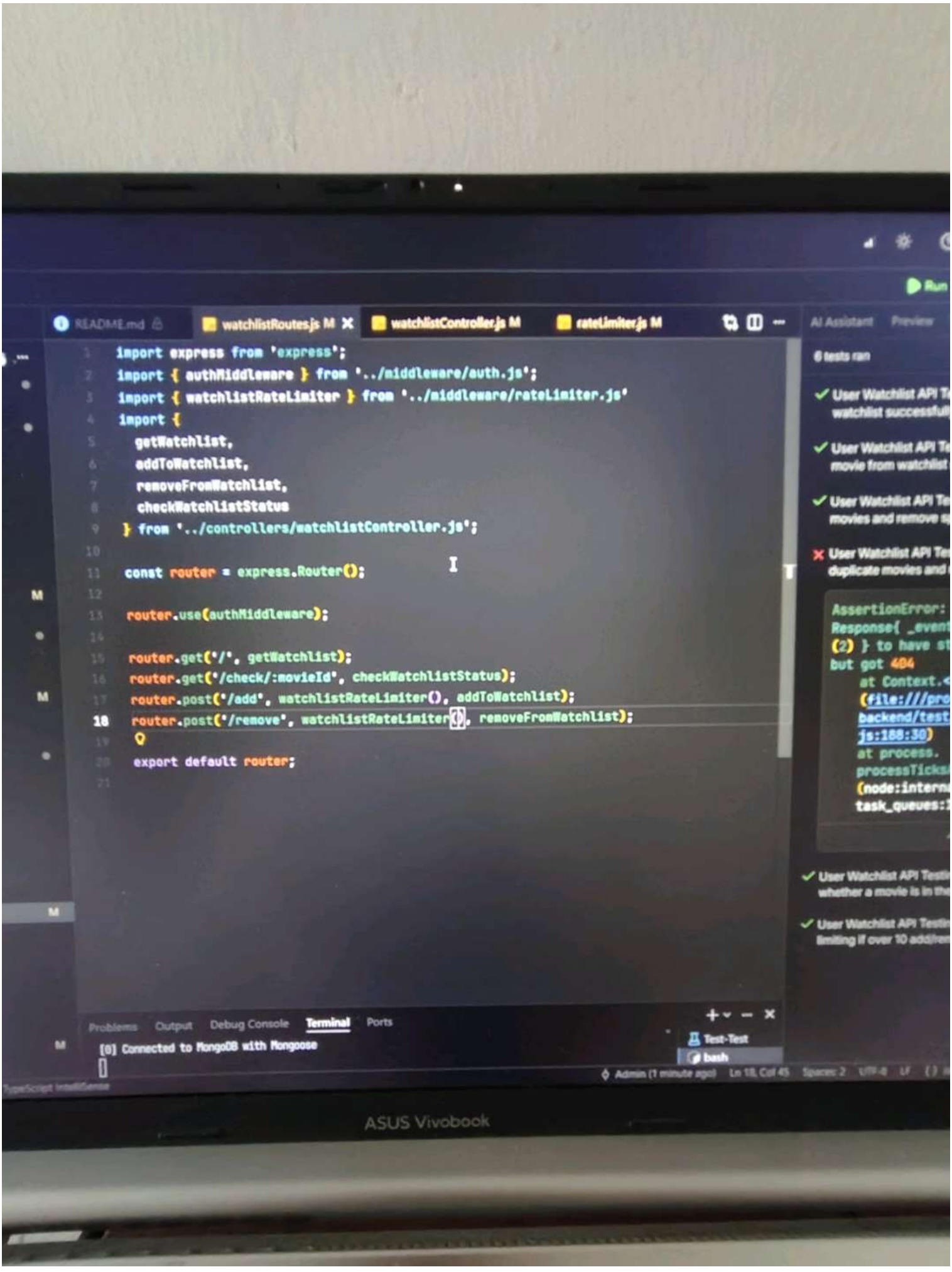

README.md
watchlistRoutes.js M
watchlistController.js M
rateLimiter.js M
import mongoose from 'mongoose';
import Watchlist from '../models/Watchlist.js';
import Movie from '../models/Movie.js';
export const getWatchlist = async (req, res) => {
try {
const userId = req.userId;
const watchlistItems = await Watchlist.find({user: userId}).populate('movie');
const movies = watchlistItems
.filter((item) => item.movie)
.map((item) => item.movie);
res.status(200).json(movies);
} catch (error) {
console.error('Get watchlist error:', error);
res.status(500).json({ message: 'Server error' });
}
};
export const addToWatchlist = async (req, res) => {
try {
const userId = req.userId;
const { movieId } = req.body;
if (!movieId) {
return res.status(400).json({ message: 'Movie ID is required' });
}
if (!mongoose.Types.ObjectId.isValid(movieId)) {
Problems
Output
Debug Console
Terminal
Ports
[0] Connected to MongoDB with Mongoose
Test-Test
bash
ASUS Vivobook


README.md
watchlistRoutes.js M
watchlistController.js M
rateLimiter.js M
export const addToWatchlist = async (req, res) => {
try {
const userId = req.userId;
const { movieId } = req.body;
if (!movieId) {
return res.status(400).json({ message: 'Movie ID is required' });
}
if (!mongoose.Types.ObjectId.isValid(movieId)) {
return res.status(400).json({ message: 'Invalid movie ID format' });
}
const movie = await Movie.findById(movieId);
if (!movie) {
return res.status(404).json({ message: 'Movie not found' });
}
//check if movie is already present in watchList
const present = await Watchlist.findOne({user: userId, movie: movieId});
if(present){
res.status(404).json({ message: 'Movie already in watchlist' });
}
//if not present, then add the movie to watchlist
await Watchlist.create({ user: userId, movie: movieId });
res.status(201).json({ message: 'Movie added to watchlist successfully' });
} catch (error) {
console.error('Add to watchlist error:', error);
Problems
Output
Debug Console
Terminal
Ports
[0] Connected to MongoDB with Mongoose
Test-Test
bash
ASUS Vivobook


README.md
watchlistRoutes.js M
watchlistController.js M
rateLimiter.js M
export const addToWatchlist = async (req, res) => {
return res.status(404).json({ message: 'Movie not found' });
}
//check if movie is already present in watchList
const present = await Watchlist.findOne({user: userId, movie: movieId});
if(present){
res.status(404).json({ message: 'Movie already in watchlist' });
}
//if not present, then add the movie to watchlist
await Watchlist.create({ user: userId, movie: movieId });
res.status(201).json({ message: 'Movie added to watchlist successfully' });
} catch (error) {
console.error('Add to watchlist error:', error);
// Handle duplicate key error (though we check above, this is a safety net)
if (error.code === 11000) {
return res.status(400).json({ message: 'Movie already in watchlist' });
}
res.status(500).json({ message: 'Server error' });
}
};
export const removeFromWatchlist = async (req, res) => {
try {
const userId = req.userId;
const { movieId } = req.body;
Problems
Output
Debug Console
Terminal
Ports
[0] Connected to MongoDB with Mongoose
Test-Test
bash
AI Assistant
6 tests ran
User Watchlist API
watchlist success
User Watchlist API
movie from watchl
User Watchlist API
movies and remove
User Watchlist API
duplicate movies a
AssertionError
Response{ _eve
(2) } to have
but got 404
at Context.
(file:///pr
backend/tes
js:188:30)
at process.
processTicks
(node:intern
task_queues:
User Watchlist API Testi
whether a movie is in th
User Watchlist API Testi
limiting if over 10 addres
Admin (23 minutes ago)
Ln 107, Col 39
Spaces: 2
UTF-8
ASUS Vivobook


README.md
watchlistRoutes.js M
watchlistController.js M
rateLimiter.js M
};
export const removeFromWatchlist = async (req, res) => {
try {
const userId = req.userId;
const { movieId } = req.body;
if (!movieId) {
return res.status(400).json({ message: 'Movie ID is required' });
}
if (!mongoose.Types.ObjectId.isValid(movieId)) {
return res.status(400).json({ message: 'Invalid movie ID format' });
}
//Delete the movie from the watchList
const deletem = await Watchlist.findOneAndDelete({ user: userId, movie: movieId});
//check if movie existed or not
if(!deletem){
return res.status(404).json({ message: 'Movie not found in watchlist'});
}
//else return that it is deleted
res.json({ message: 'Movie removed from watchlist successfully' });
} catch (error) {
console.error('Remove from watchlist error:', error);
res.status(500).json({ message: 'Server error' });
}
};
AI Assistant
6 tests ran
User Watchlist API
User Watchlist API
User Watchlist API
User Watchlist API
AssertionError
but got 404
at Context.
(file:///
backend/te
js:188:30)
at process.
processTick
(node:inter
task_queues:
User Watchlist API Test
User Watchlist API Test
Problems
Output
Debug Console
Terminal
Ports
[0] Connected to MongoDB with Mongoose
Test-Test
bash
ASUS Vivobook


README.md
watchlistRoutes.js M
watchlistController.js M
rateLimiter.js M
export const removeFromWatchlist = async (req, res) => {
res.status(500).json({ message: 'Server error' });
};
export const checkWatchlistStatus = async (req, res) => {
try {
const userId = req.userId;
const { movieId } = req.params;
if (!mongoose.Types.ObjectId.isValid(movieId)) {
return res.status(400).json({ message: 'Invalid movie ID format' });
}
//first check if req movie is present in watchlist or not
const isInWatchlist = await Watchlist.findOne({ user: userId, movie: movieId });
//we have to return boolean value
res.status(200).json({ inWatchlist: !!isInWatchlist });
} catch (error) {
console.error('Check watchlist status error:', error);
res.status(500).json({ message: 'Server error' });
}
};
Run
AI Assistant
Preview
6 tests ran
User Watchlist API Tes
watchlist successfully
User Watchlist API Test
movie from watchlist su
User Watchlist API Test
movies and remove spe
User Watchlist API Testi
duplicate movies and re
AssertionError: ex
Response{ _events:
(2) } to have stat
but got 404
at Context.<an
(file:///proje
backend/test/a
js:188:30)
at process.
processTicksAnd
(node:internal/
task_queues:105
User Watchlist API Testing sh
whether a movie is in the wa
User Watchlist API Testing sh
limiting if over 10 add/remove
Problems
Output
Debug Console
Terminal
Ports
[0] Connected to MongoDB with Mongoose
Test-Test
bash
Admin (23 minutes ago)
Ln 107, Col 39
Spaces: 2
UTF-8
LF
ASUS Vivobook


README.md
watchlistRoutes.js M
watchlistController.js M
rateLimiter.js M
import express from 'express';
import { authMiddleware } from '../middleware/auth.js';
import { watchlistRateLimiter } from '../middleware/rateLimiter.js'
import {
getWatchlist,
addToWatchlist,
removeFromWatchlist,
checkWatchlistStatus
} from '../controllers/watchlistController.js';
const router = express.Router();
router.use(authMiddleware);
router.get('/', getWatchlist);
router.get('/check/:movieId', checkWatchlistStatus);
router.post('/add', watchlistRateLimiter(), addToWatchlist);
router.post('/remove', watchlistRateLimiter(), removeFromWatchlist);
export default router;
AI Assistant
Preview
6 tests ran
User Watchlist API
User Watchlist API
User Watchlist API
User Watchlist API
AssertionError:
but got 404
Problems
Output
Debug Console
Terminal
Ports
[0] Connected to MongoDB with Mongoose
Test-Test
bash
ASUS Vivobook